

Lógica y Representación 1

Condicionales múltiples

Ingeniería de Sistemas

Universidad de Antioquia

Método de Polya - Resumen



PROBLEMA



ALGORITMO



1

Entender el problema

2

Diseñar un plan

3

Implementar el plan

4

Revisar el plan

Entrada, procesamiento y salida

Los programas de computadora realizan estos tres pasos:

1. Se recibe la **entrada**.
2. Se realiza un **proceso** sobre la entrada.
3. Se genera la **salida**



Inputs



Process

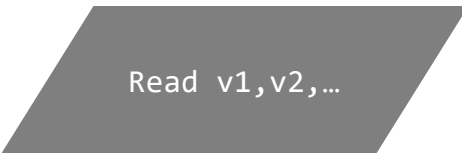
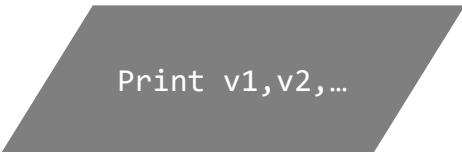


Outputs



Tabla resumen

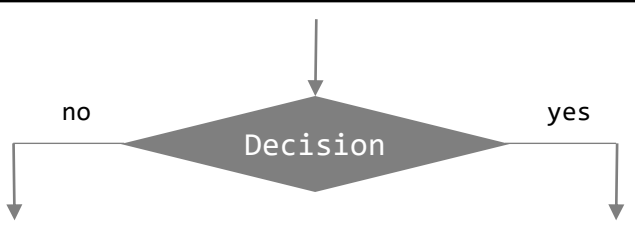
La siguiente tabla muestra la representación equivalente de las piezas esenciales para construir código secuencial

Tipo	Diagrama de bloques	Pseudocodigo	Python
Entrada 		Leer(v1,v2,...)	<pre>v1 = input(...) v2 = input(...)</pre>
Salida 		Escribir(v1,v2,...)	<pre>print(v1,v2,...)</pre>
Proceso 		$c = a + b$	$c = a + b$

En Python, `input()` siempre devuelve texto (`str`); si el valor se va a usar en una operación numérica, debe convertirse con `int()` o `float()` (por ejemplo: `v1 = float(input(...))`).

Tabla resumen

La siguiente tabla, agrega la nueva el nuevo bloque que permite implementar toma de decisiones en los programas.

Diagrama de bloques	Pseudocodigo	Python
	Si (condicion) Entonces instrucciones verdad Sino instrucciones falsa Fin_Si	<pre>if (condicion): instrucciones verdad else: instrucciones falso</pre>

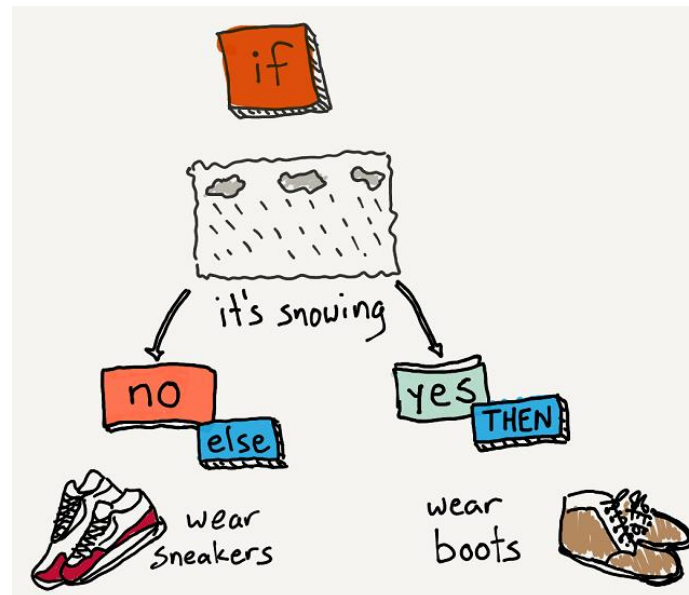


Imagen tomada de CS Intro ([link](#))

Casos en los problemas de decisión lógica

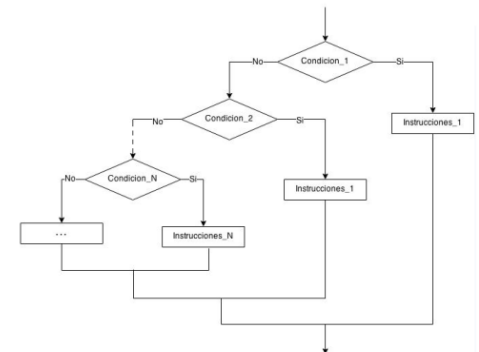
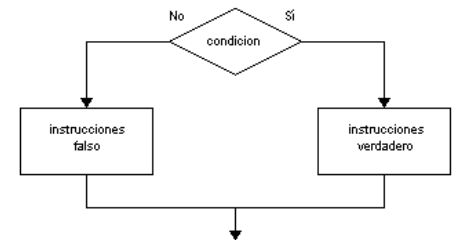
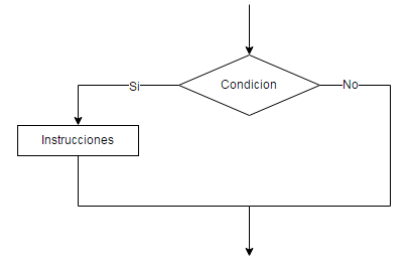


**Alternativas en los
problemas de decisión
lógica**

Alternativa simple

Alternativa doble

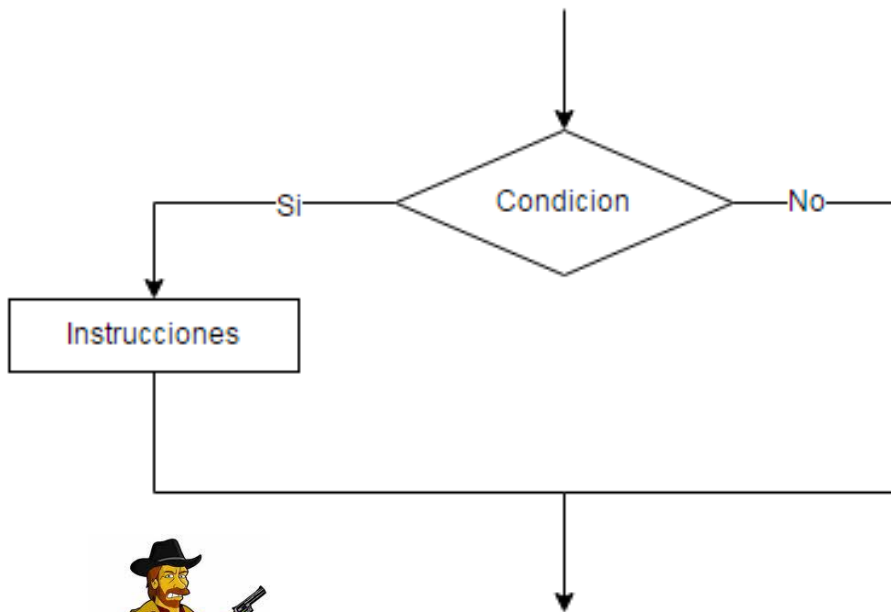
Alternativa múltiple



Alternativa simple

En los problemas de este tipo la condición **falsa** no posee instrucciones.

Diagrama de flujo



Pseudocódigo

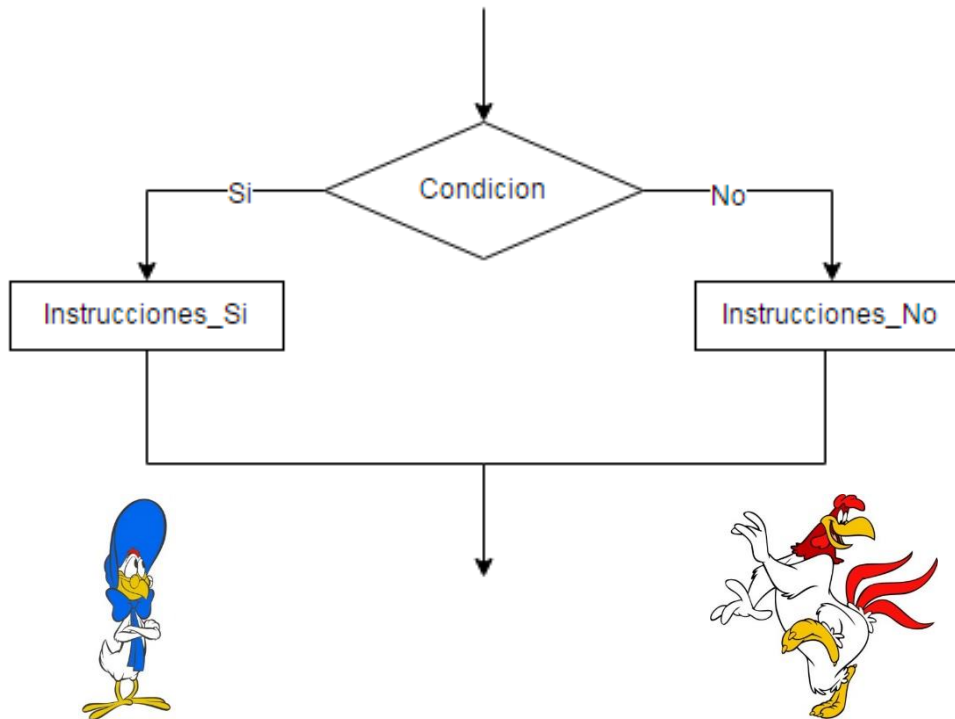
```
Si (condicion) Entonces  
    instrucciones condicion verdadera  
Fin_Si
```

Python

```
if (condicion):  
    instrucciones condicion verdadera
```

Alternativa doble

En los problemas de este tipo, ambos casos (**Falso** y **Verdadero**) poseen instrucciones asociadas.



Pseudocódigo

```
Si (condicion) Entonces  
    instrucciones condicion verdadera  
Sino  
    instrucciones condición falsa  
Fin_Si
```

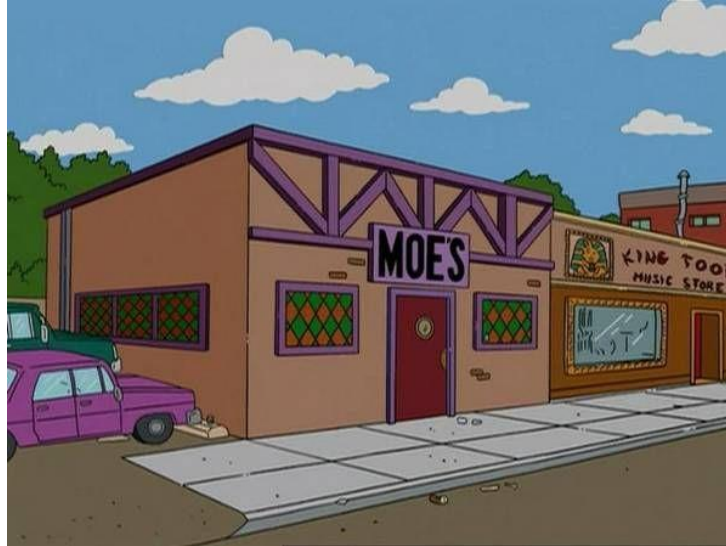
Python

```
if (condicion):  
    instrucciones condicion verdadera  
else:  
    instrucciones condición falsa
```


Alternativa simple

Ejemplo 1:

Moe con el fin de incentivar su negocio organizo una noche de solos y solas. Los precios que fijo fueron de \$10000 para los hombres y de la mitad de este valor para las mujeres. Hacer un programa que de acuerdo al sexo de la persona que asista muestre el precio de la entrada al bar de Moe.



Alternativa doble

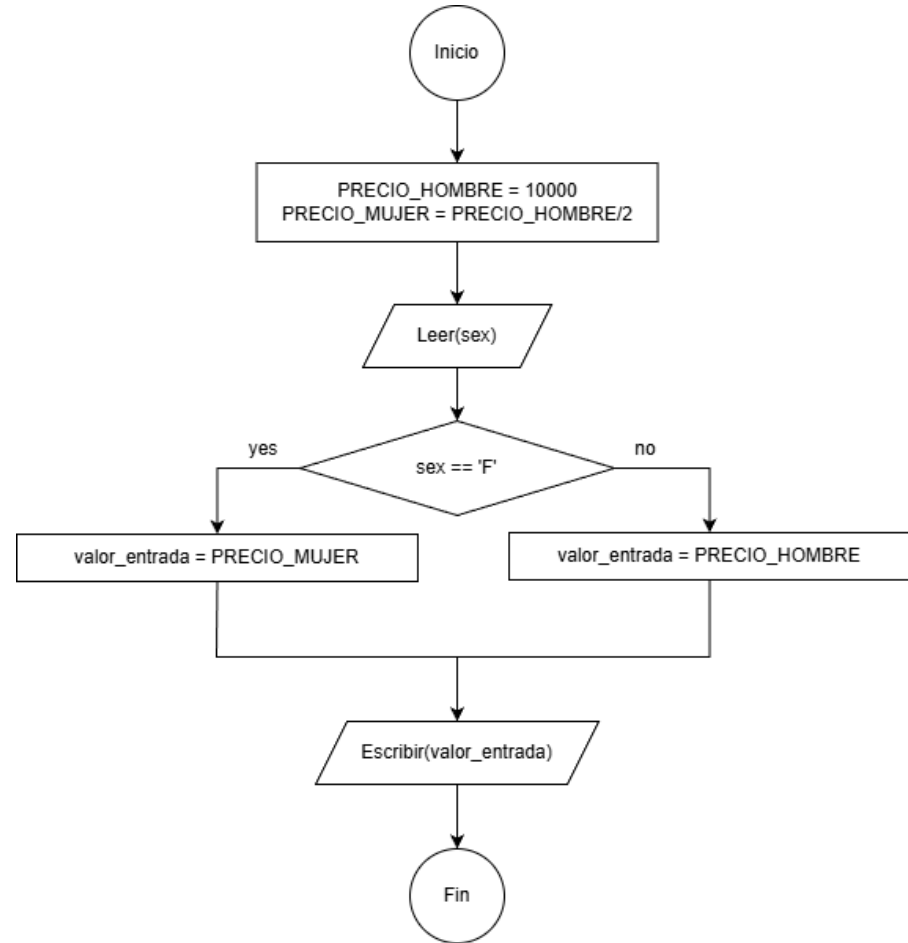
Pseudocódigo

```
Inicio
  PRECIO_HOMBRE = 10000
  PRECIO_MUJER = PRECIO_HOMBRE/2
  Leer(sex)
  Si (sex == 'F') Entonces
    valor_entrada = PRECIO_MUJER
  Sino
    valor_entrada = PRECIO_HOMBRE
  Fin_Si
  Escribir(valor_entrada)
Fin
```

Python

```
PRECIO_HOMBRE = 10000
PRECIO_MUJER = PRECIO_HOMBRE / 2
sex = input('Digite el sexo (F/M): ')
if sex == 'F':
    valor_entrada = PRECIO_MUJER
else:
    valor_entrada = PRECIO_HOMBRE
print(f'Valor de la entrada: ${valor_entrada}')
```

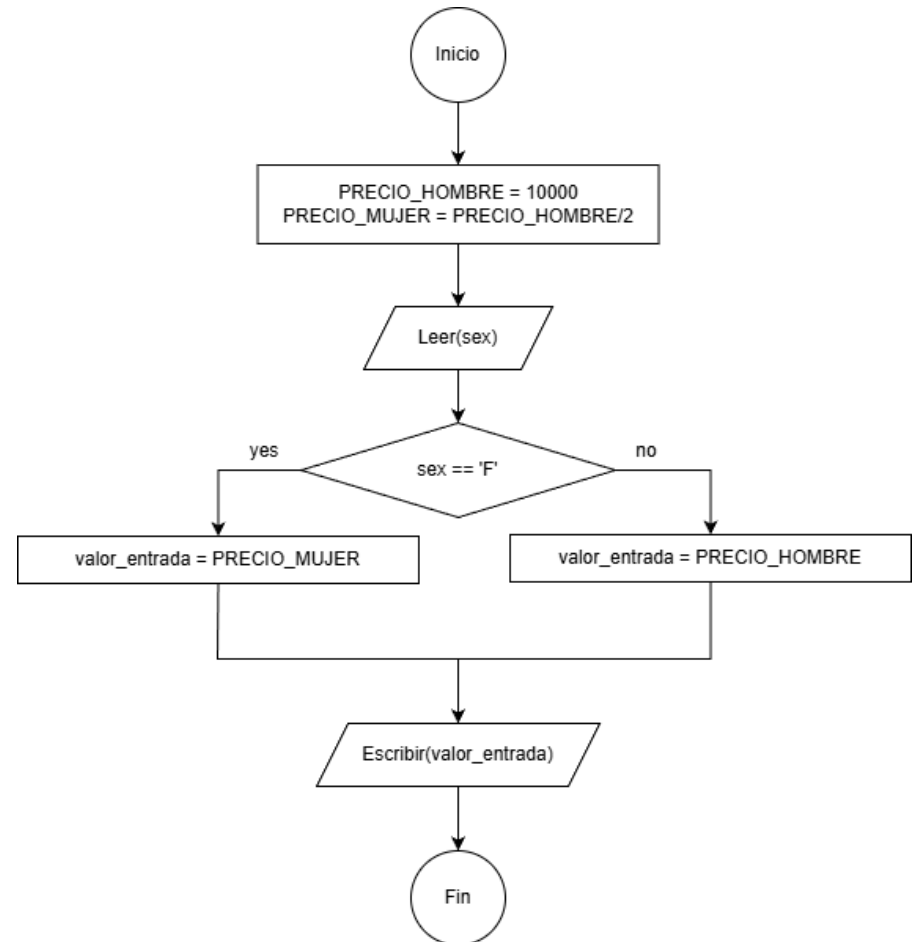
[Link de ejecución](#)



Alternativa doble

sex	valor_entrada
"M"	10000
"F"	5000
"f"	10000
"x"	10000

sex	val_entrada

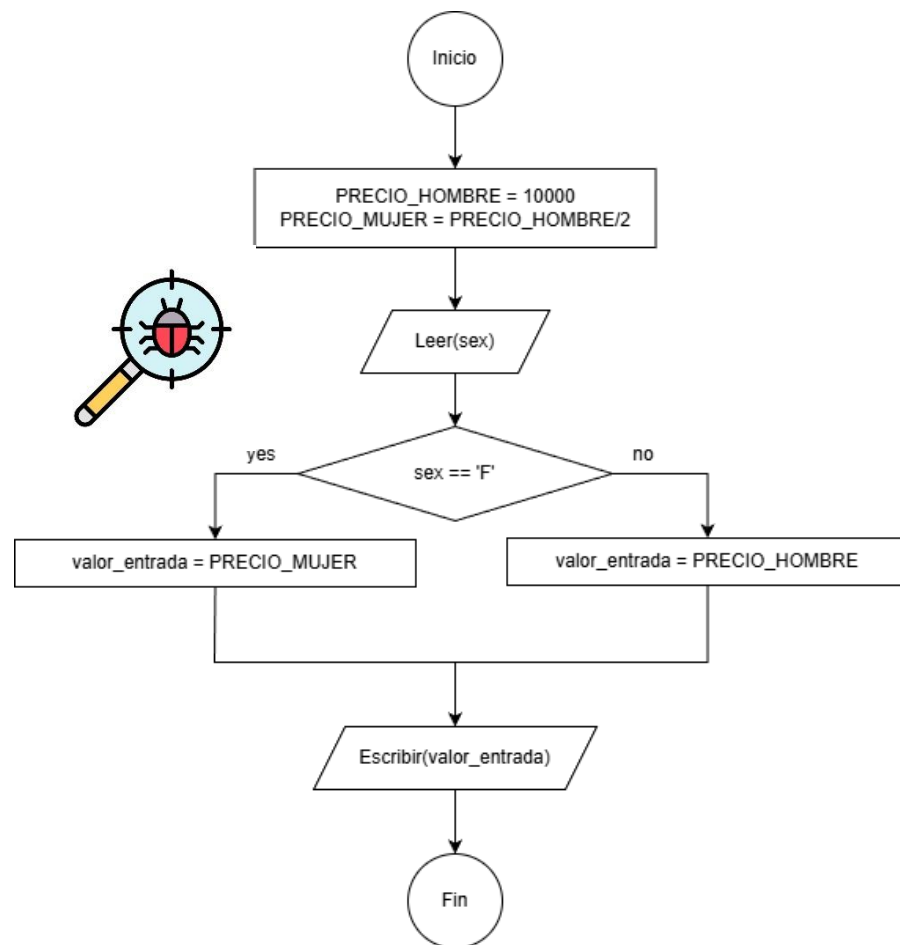


Alternativa doble

ID	valor_entrada
"M"	10000
"F"	5000
"f"	10000
"x"	10000

Resultados esperados: Si hacemos la prueba de escritorio, el algoritmo tal y como esta planteado tiene problemas en la implementación lógica (solo funciona como se espera para las letras mayúsculas "M" o "F") pero en otros casos no arrojan los resultados correctos.

1. Letras minúsculas para el genero.
2. Letras diferentes para el genero.
3. Adaptación a la inclusividad: En la actualidad el genero no es solo masculino ni femenino, hay diversidad.



Alternativa doble

Ejemplo 2:

En una empresa le dan a los empleados un subsidio de transporte si el sueldo base de estos es menor que el salario mínimo (\$2000000), este subsidio es el 30% del sueldo base. Hacer un algoritmo que calcule el salario neto de un empleado (sueldo base mas prestaciones si el empleado tiene derecho a estas). Los datos de entrada son la cedula y el sueldo base.



Alternativa doble

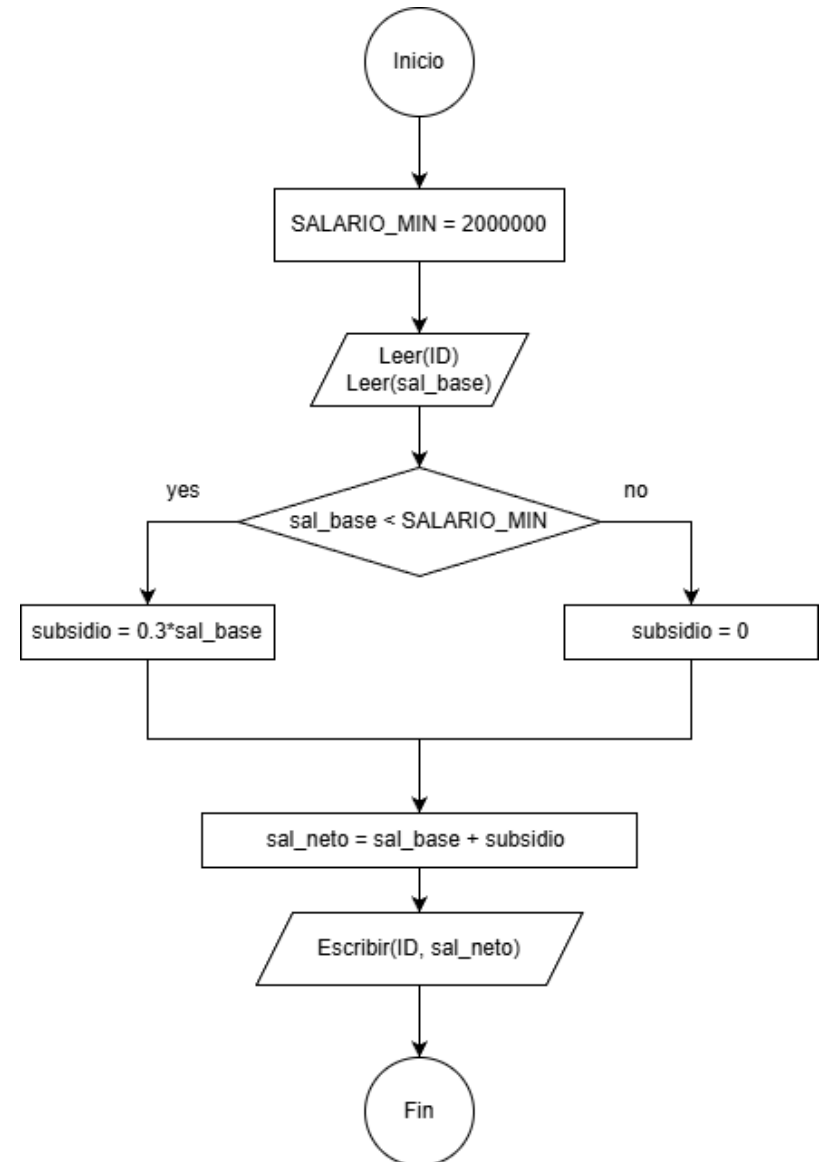
Pseudocódigo

```
Inicio
    SALARIO_MIN = 2000000
    Leer(ID)
    Leer(sal_base)
    Si (sal_base < 2000000) Entonces
        subsidio = 0.3*sal_base
    Sino
        subsidio = 0
    Fin_Si
    sal_neto = sal_base + subsidio
    Escribir(ID, sal_neto)
Fin
```

Python

```
SALARIO_MIN = 2000000
ID = input("Digite la cedula: ")
sal_base = int(input("Digite el salario base: "))
if(sal_base < SALARIO_MIN):
    subsidio = 0.3*sal_base
else:
    subsidio = 0
sal_neto = sal_base + subsidio
print(ID + ' le corresponde como pago $', \
      sal_neto, end = "")
```

[Link de ejecución](#)



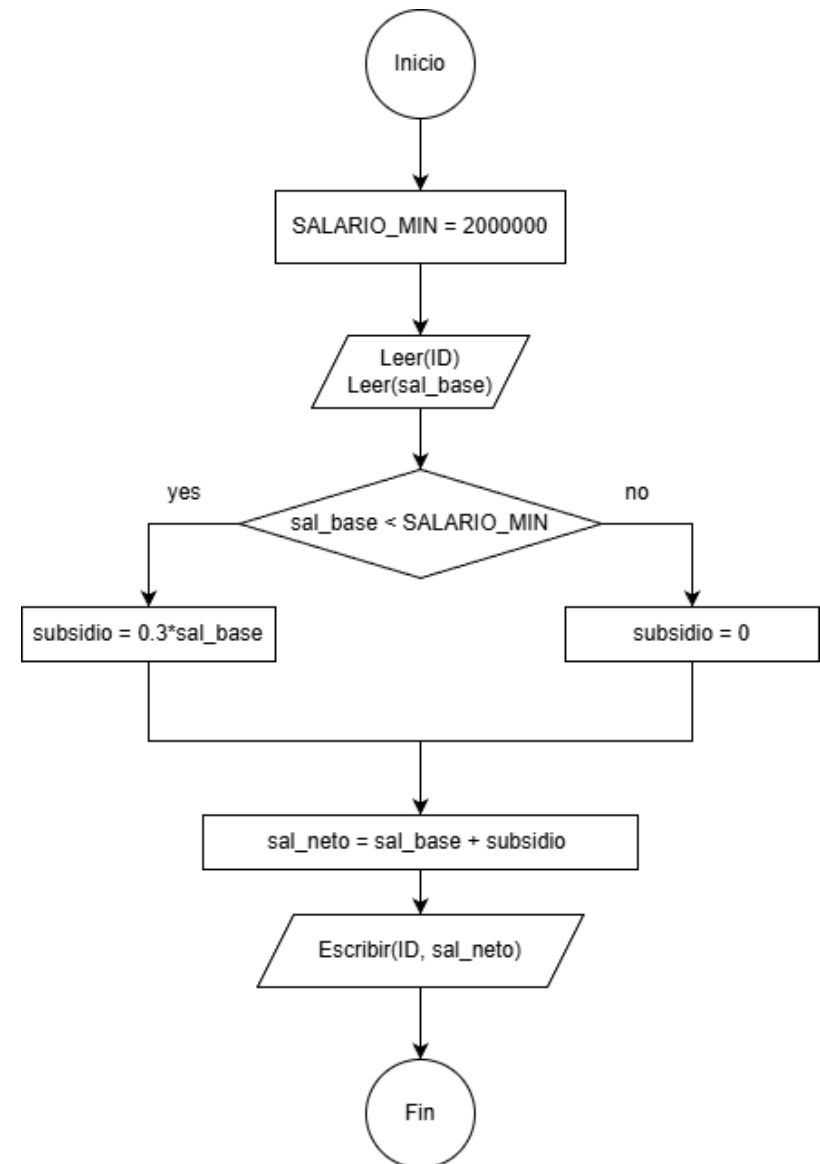
Alternativa doble

Resultados esperados

ID	sal_base	salNeto
"0000"	1000000	1300000
"0001"	3000000	3000000
"0002"	-3000000	-3900000

Prueba de escritorio:

ID	sal_base	subsidio	sal_neto

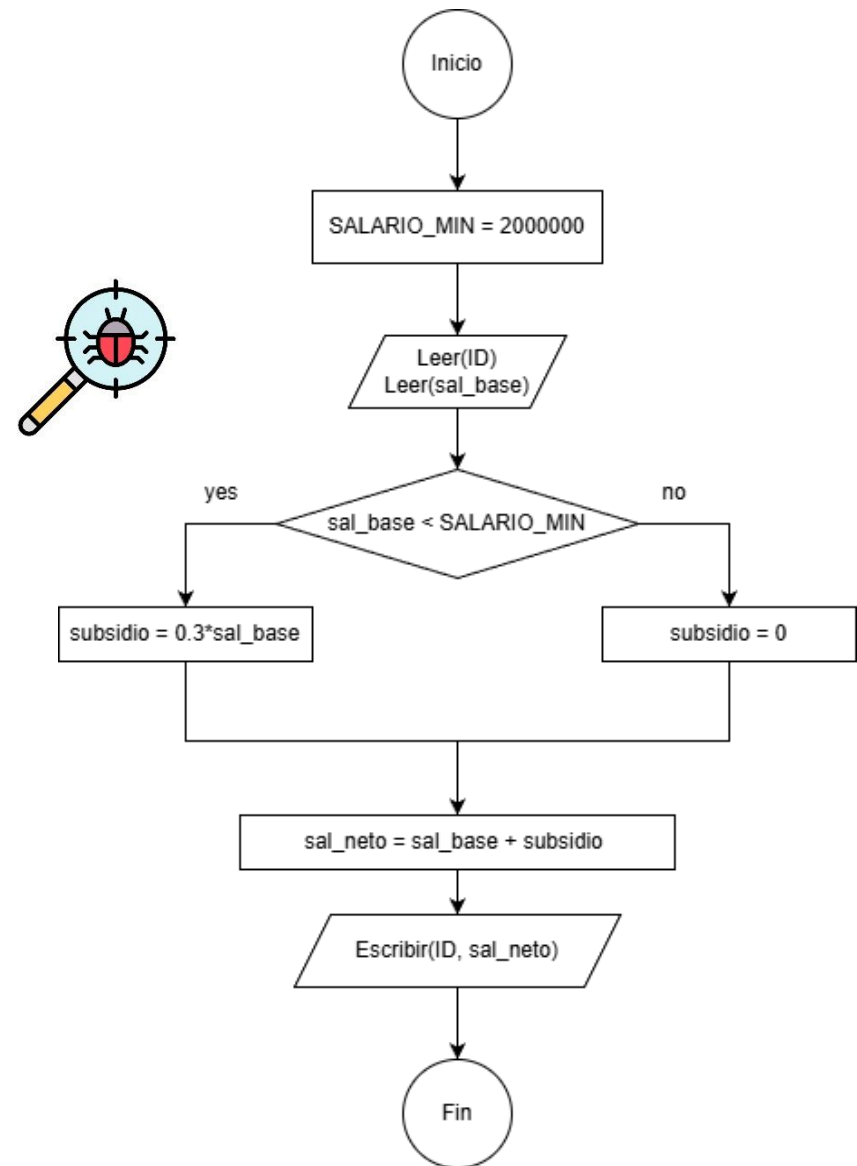


Alternativa doble

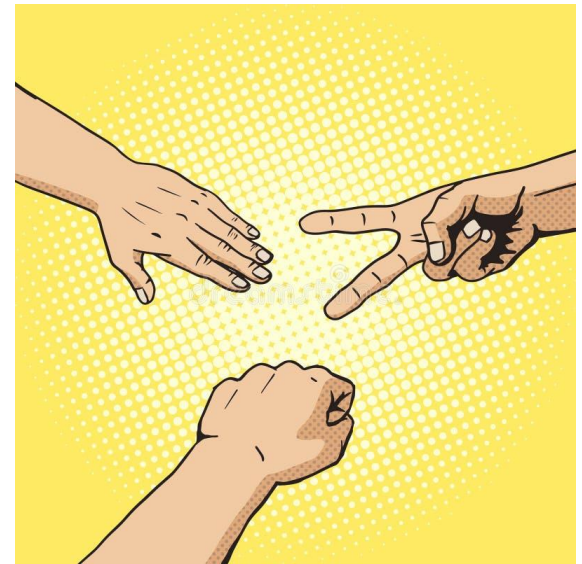
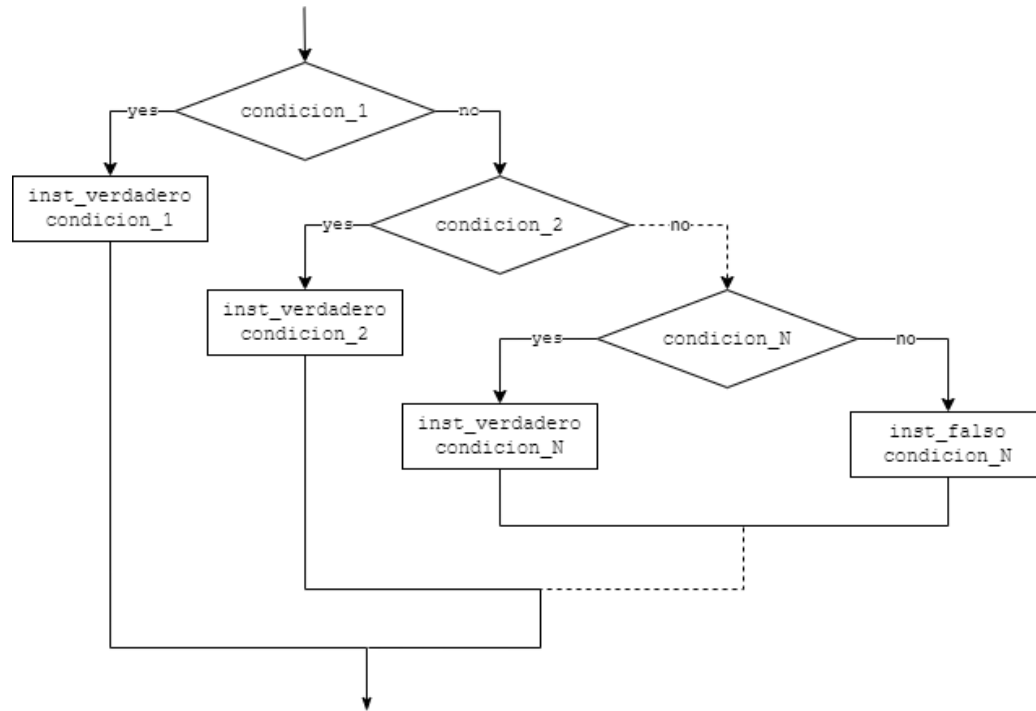
Resultados esperados

ID	sal_base	salNeto
"0000"	1000000	1300000
"0001"	3000000	3000000
"0002"	-3000000	-3900000

Resultados esperados: Aunque a este nivel asumimos que el usuario usa de manera correcta el programa, en realidad, el usuario puede ingresar (por que se equivoca o por que le da la gana) valores incorrectos. Por ejemplo, este programa tiene un **error lógico grave** si el usuario ingresa un valor negativo para el salario base.

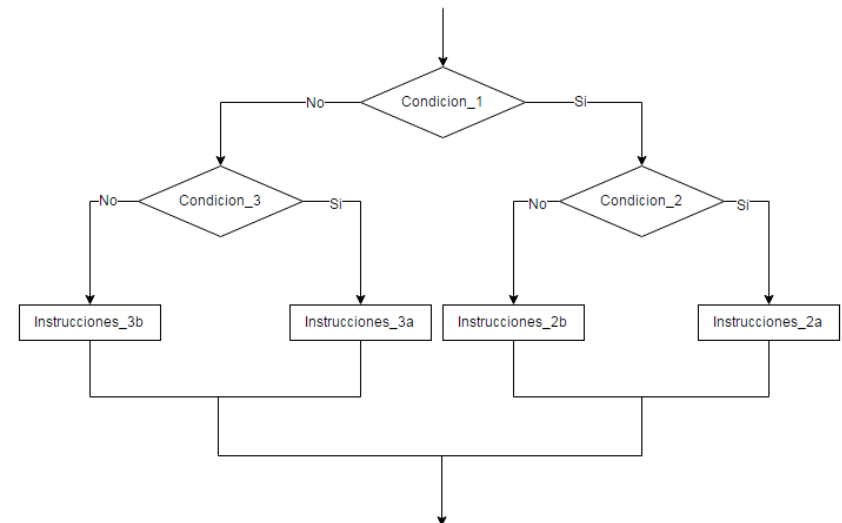
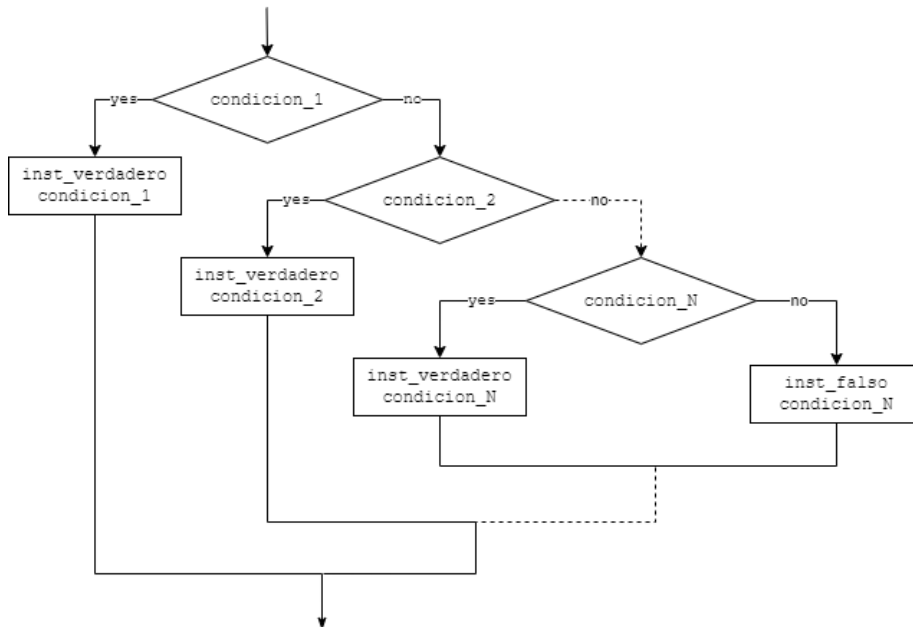


Alternativa múltiple



Alternativa múltiple

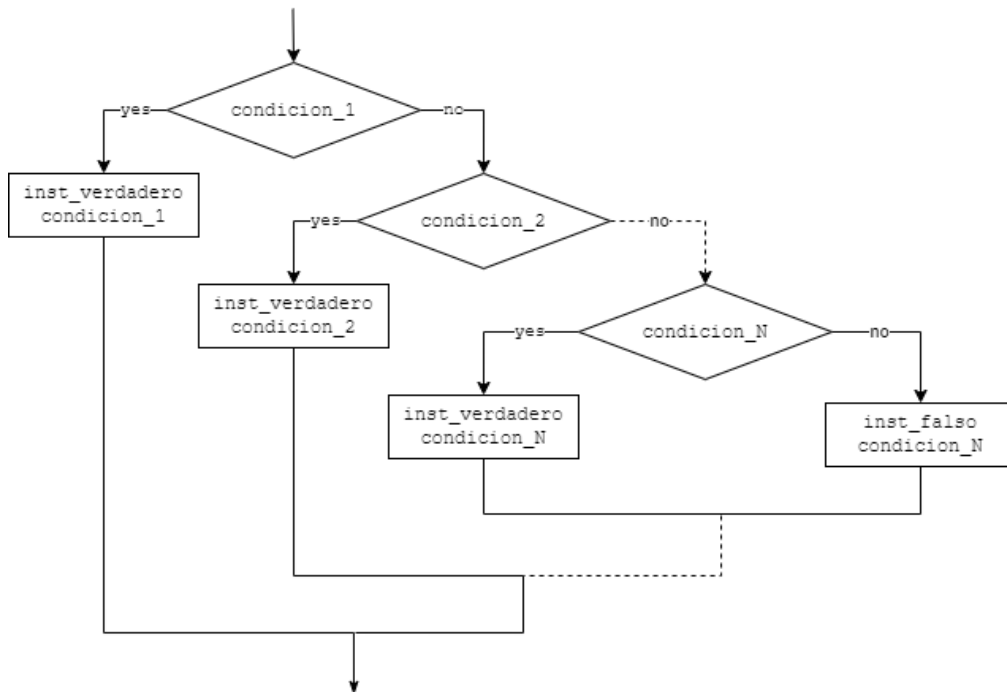
- Cuando se desea evaluar más de dos alternativas, es necesario implementar una **estructura condicional múltiple**.
- Esto se logra mediante el anidamiento de bloques básicos de estructuras condicionales (**Si...Entonces...Sino**).
- La forma de anidar depende del problema: si se evalúan varios casos de **un mismo criterio** (por ejemplo, rangos de una variable), se encadenan las condiciones una tras otra; si se evalúan **criterios distintos** según la rama que se tome, se anidan condiciones completas unas dentro de otras.



Alternativa múltiple – Mismo criterio

Veamos el caso en el que todas las condiciones evalúan el **mismo criterio**, por ejemplo, distintos rangos de una misma variable (como una escala de notas)

Diagrama de flujo



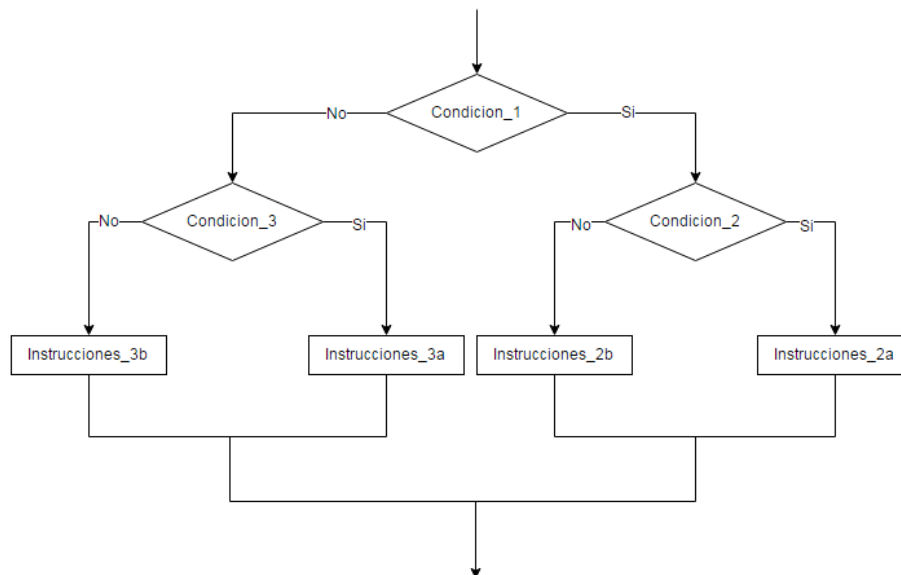
Pseudocódigo

```
Si (condicion_1) Entonces  
    condicion_1 verdad  
Sino  
    Si (condicion_2) Entonces  
        condicion_2 verdad  
    Sino  
        Si (condicion_3) Entonces  
            condicion_3 verdad  
        Sino  
            ...  
            Si (condicion_n) Entonces  
                condicion_n verdad  
            Sino  
                condicion_n falso  
            Fin_Si  
        ...  
    Fin_Si  
Fin_Si
```

Alternativa múltiple – Diferente criterio

Veamos el caso en el que cada rama evalúa un **criterio distinto**, por ejemplo, un umbral que cambia según la decisión anterior (como el sexo y luego un límite de edad diferente para cada uno).

Diagrama de flujo



Pseudocódigo

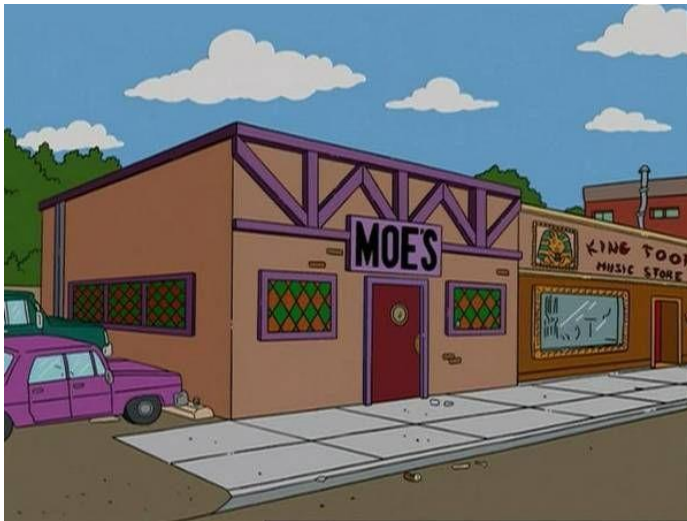
```
Si (condicion_1) Entonces
  Si (condicion_3) Entonces
    condicion_3a verdad
  Sino
    condicion_3b
  Fin_Si
Sino
  Si (condicion_2) Entonces
    condicion_2a
  Sino
    condicion_2b
  Fin_Si
Fin_Si
```

Alternativa múltiple

Ejemplo 1 (Mejorando el problema):

Moe con el fin de incentivar su negocio organizo una noche de solos y solas. Los precios que fijo fueron de \$10000 para los hombres y de la mitad de este valor para las mujeres. Hacer un programa que de acuerdo al sexo de la persona que asista muestre el precio de la entrada al bar de Moe.

Nota: Resuelva el bug del programa



Alternativa doble

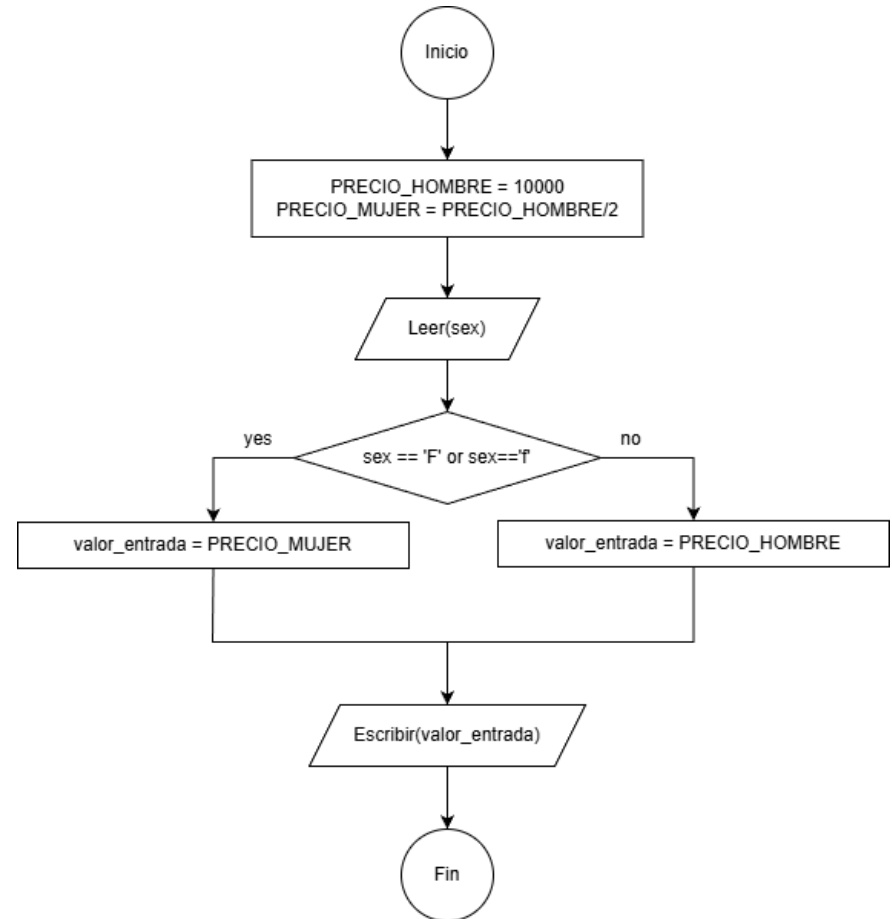
Pseudocódigo

```
Inicio
    PRECIO_HOMBRE = 10000
    PRECIO_MUJER = PRECIO_HOMBRE/2
    Leer(sex)
    Si ((sex == 'F') or (sex == 'f')) Entonces
        valor_entrada = PRECIO_MUJER
    Sino
        valor_entrada = PRECIO_HOMBRE
    Fin_Si
    Escribir(valor_entrada)
Fin
```

Python

```
PRECIO_HOMBRE = 10000
PRECIO_MUJER = PRECIO_HOMBRE / 2
sex = input('Digite el sexo (F/M): ')
if ((sex == 'F') or (sex == 'f')):
    valor_entrada = PRECIO_MUJER
else:
    valor_entrada = PRECIO_HOMBRE
print(f'Valor de la entrada: ${valor_entrada}')
```

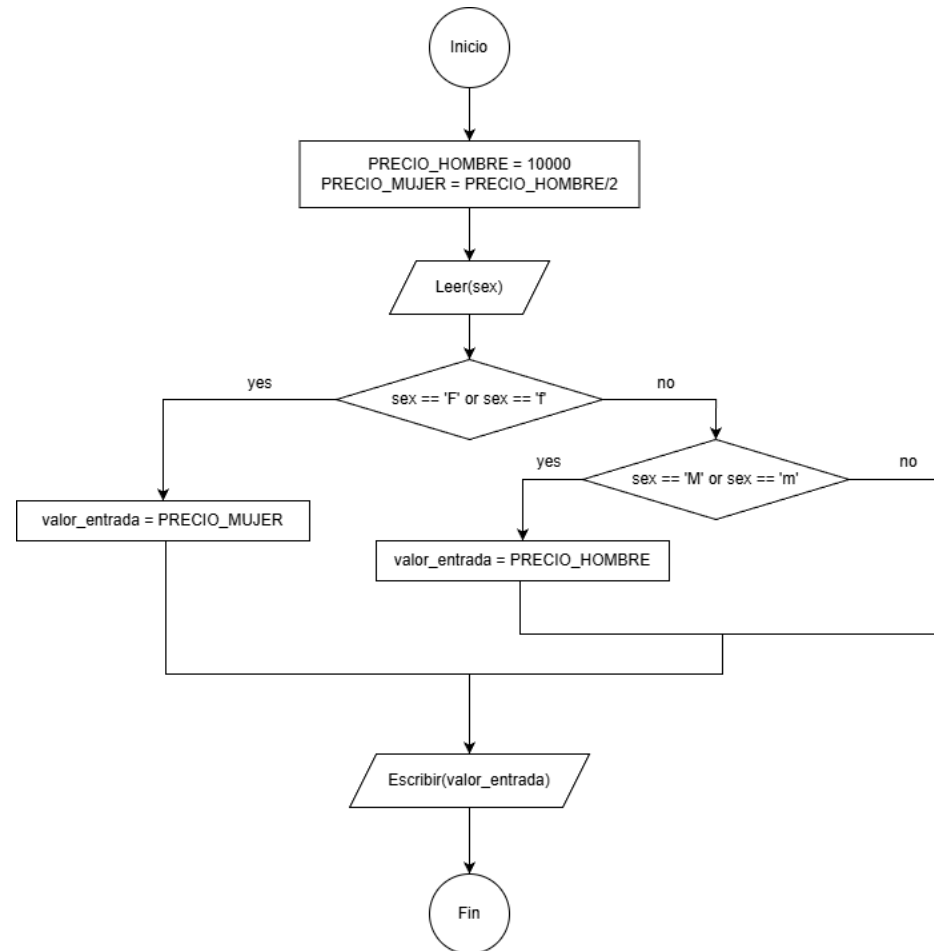
[Link de ejecución](#)



Alternativa múltiple

Pseudocódigo

```
Inicio
    PRECIO_HOMBRE = 10000
    PRECIO_MUJER = PRECIO_HOMBRE/2
    Leer(sex)
    Si ((sex == 'F') or (sex == 'f')) Entonces
        valor_entrada = PRECIO_MUJER
    Sino
        Si ((sex == 'M') or (sex == 'm')) Entonces
            valor_entrada = PRECIO_HOMBRE
        Fin_Si
    Fin_Si
    Escribir(valor_entrada)
Fin
```

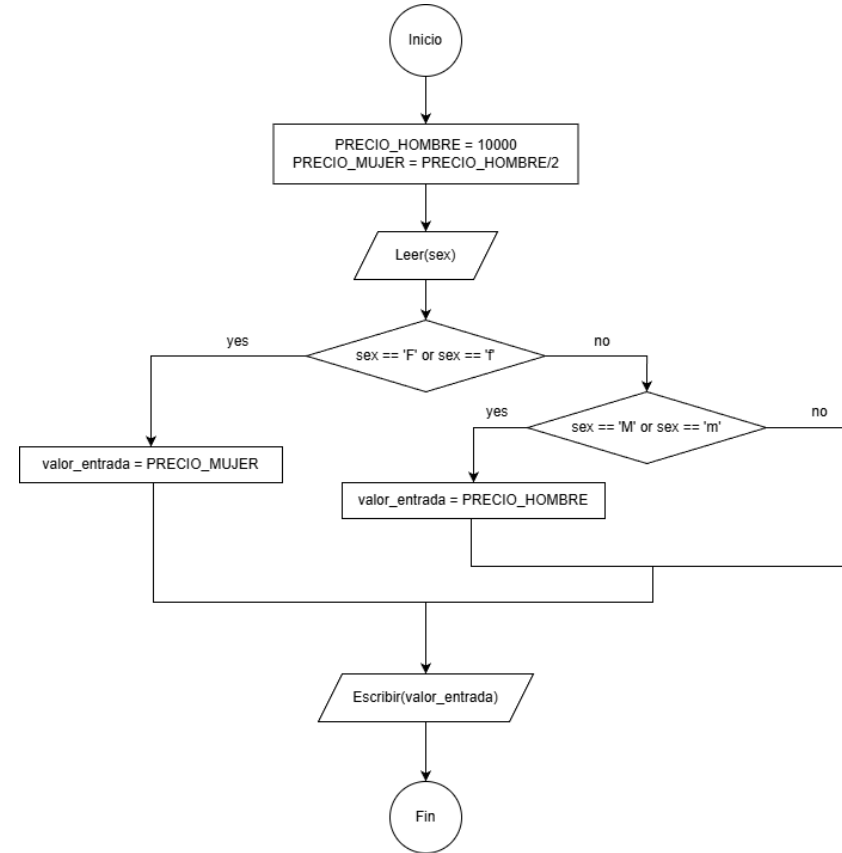


Alternativa múltiple

Python

```
PRECIO_HOMBRE = 10000
PRECIO_MUJER = PRECIO_HOMBRE / 2
sex = input('Digite el sexo (F/M): ')
if ((sex == 'F') or (sex == 'f')):
    valor_entrada = PRECIO_MUJER
else:
    if ((sex == 'M') or (sex == 'm')):
        valor_entrada = PRECIO_HOMBRE
    print(f'Valor de la entrada: ${valor_entrada}')
```

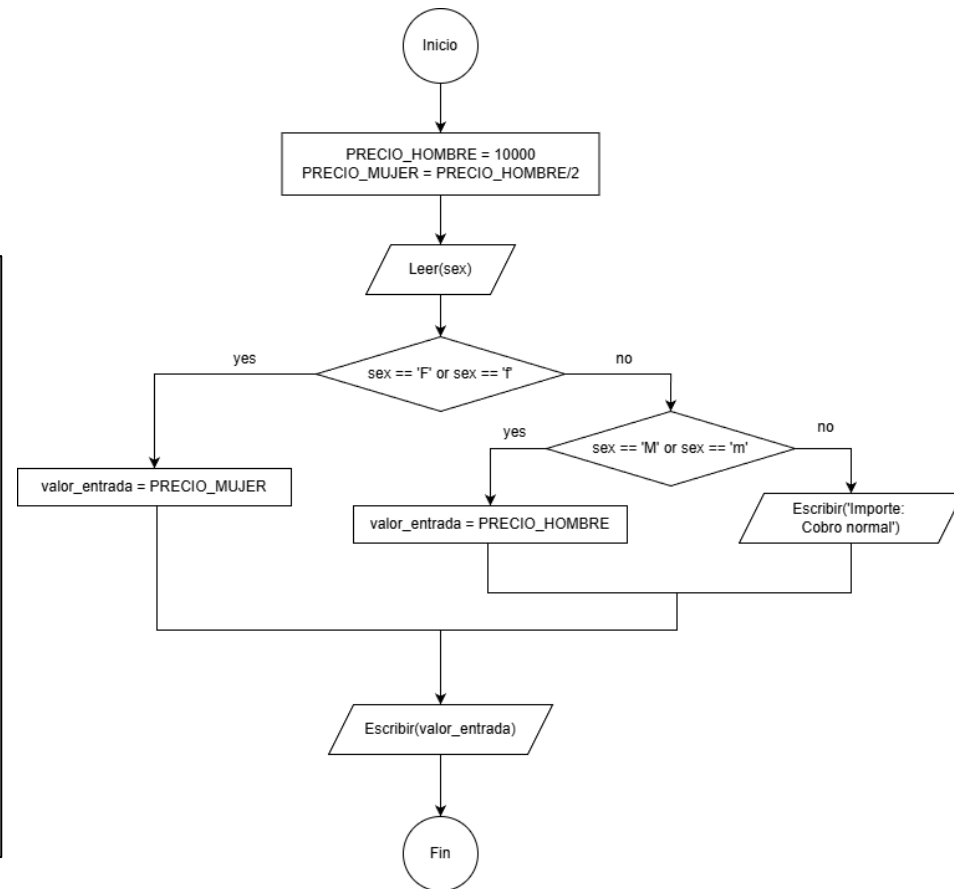
[Link de ejecución](#)



Alternativa múltiple

Pseudocódigo

```
Inicio
  PRECIO_HOMBRE = 10000
  PRECIO_MUJER = PRECIO_HOMBRE/2
  Leer(sex)
  Si ((sex == 'F') or (sex == 'f')) Entonces
    valor_entrada = PRECIO_MUJER
  Sino
    Si ((sex == 'M') or (sex == 'm')) Entonces
      valor_entrada = PRECIO_HOMBRE
    Sino
      Escribir('Importe: Cobro normal')
    Fin_Si
  Fin_Si
  Escribir(valor_entrada)
Fin
```

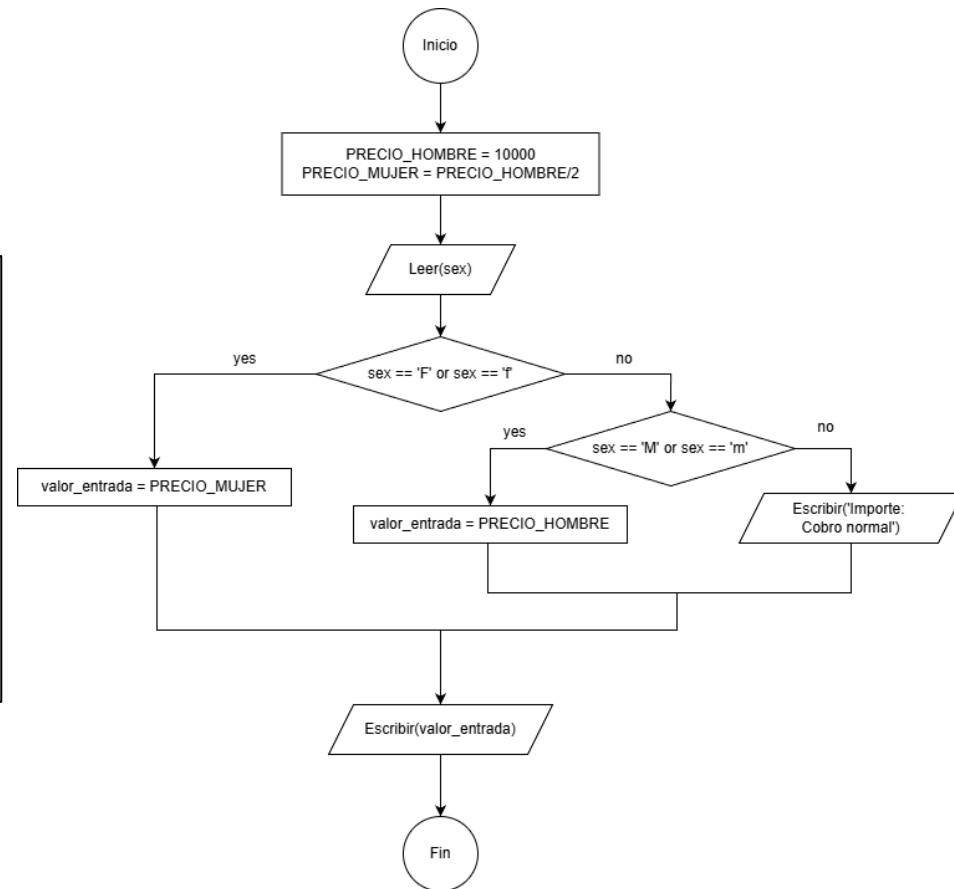


Alternativa múltiple

Python

```
PRECIO_HOMBRE = 10000
PRECIO_MUJER = PRECIO_HOMBRE / 2
sex = input('Digite el sexo (F/M): ')
if ((sex == 'F') or (sex == 'f')):
    valor_entrada = PRECIO_MUJER
else:
    if ((sex == 'M') or (sex == 'm')):
        valor_entrada = PRECIO_HOMBRE
    else:
        print('Importe: Cobro normal')
print(f'Valor de la entrada: ${valor_entrada}')
```

[Link de ejecución](#)



Alternativa múltiple

ID	valor_entrada
"M"	10000
"F"	5000
"f"	10000
"x"	10000

sex	val_entrada

```
Inicio
    PRECIO_HOMBRE = 10000
    PRECIO_MUJER = PRECIO_HOMBRE/2
    Leer(sex)
    Si ((sex == 'F') or (sex == 'f')) Entonces
        valor_entrada = PRECIO_MUJER
    Sino
        Si ((sex == 'M') or (sex == 'm')) Entonces
            valor_entrada = PRECIO_HOMBRE
        Sino
            Escribir('Importe: Cobro normal')
        Fin_Si
    Fin_Si
    Escribir(valor_entrada)
Fin
```

Alternativa múltiple

Ejemplo 2:

En una empresa le dan a los empleados un subsidio de transporte si el sueldo base de estos es menor que el salario mínimo, este subsidio es el 30% del sueldo base. Hacer un algoritmo que calcule el salario neto de un empleado (sueldo base mas prestaciones si el empleado tiene derecho a estas). Los datos de entrada son la cedula y el sueldo base.

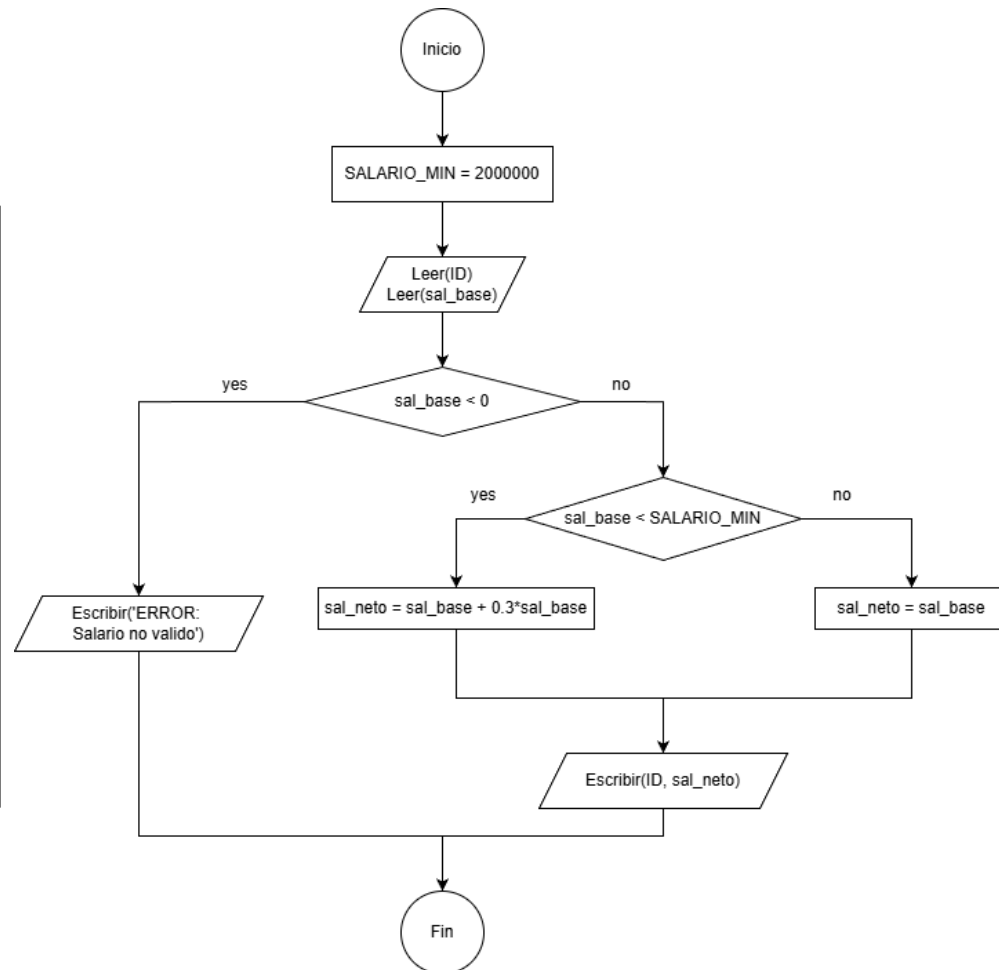
Nota: Resuelva el bug del programa.



Alternativa múltiple

Pseudocódigo

```
Inicio
  SALARIO_MIN = 2000000
  Leer(ID)
  Leer(sal_base)
  Si (sal_base < 0) Entonces
    Escribir('ERROR: Salario no valido')
  Sino
    Si (sal_base < SALARIO_MIN) Entonces
      sal_neto = sal_base + 0.3*sal_base
    Sino
      sal_neto = sal_base
    Fin_Si
    Escribir(ID, sal_neto)
  Fin_Si
Fin
```

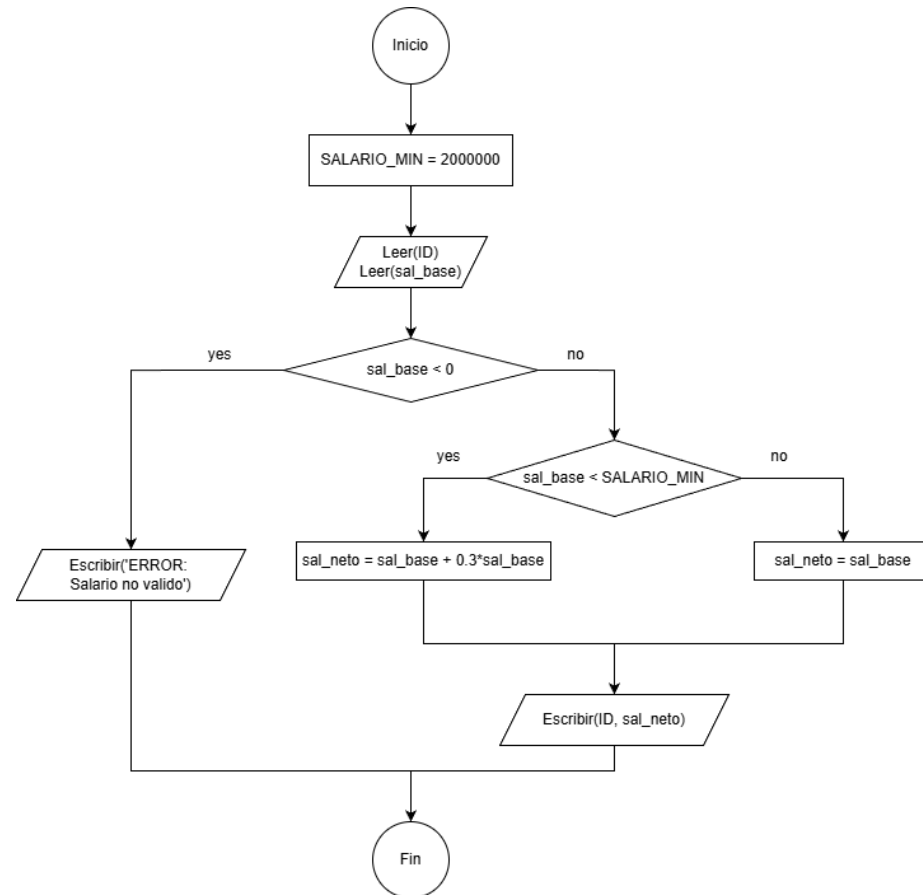


Alternativa múltiple

Python

```
SALARIO_MIN = 2000000
ID = input("Digite la cedula: ")
sal_base = int(input("Digite el salario base: "))
if(sal_base < 0):
    print('ERROR: Salario no valido')
else:
    if(sal_base < SALARIO_MIN):
        sal_neto = sal_base + 0.3*sal_base
    else:
        sal_neto = sal_base
    print(ID + ' le corresponde como pago $', \
          sal_neto, end = "")
```

[Link de ejecución](#)



Alternativa múltiple

Resultados esperados

ID	sal_base	sal_neto
"0000"	1000000	1300000
"0001"	3000000	3000000
"0002"	-3000000	----

Prueba de escritorio:

ID	sal_base	subsidio	sal_neto

```
Inicio
  SALARIO_MIN = 2000000
  Leer(ID)
  Leer(sal_base)
  Si (sal_base < 0) Entonces
    Escribir('ERROR: Salario no valido')
  Sino
    Si (sal_base < SALARIO_MIN) Entonces
      sal_neto = sal_base + 0.3*sal_base
    Sino
      sal_neto = sal_base
    Fin_Si
    Escribir(ID, sal_neto)
  Fin_Si
Fin
```

Ejemplos de repaso

3. En el bachillerato de Springfield las calificaciones se suelen calcular de acuerdo al siguiente cuadro:

Grado numérico	Grado en letra
Grado mayor o igual a 90	A
Grado menor que 90 pero mayor o igual a 80	B
Grado menor que 80 pero mayor o igual a 70	C
Grado menor que 70 pero mayor o igual a 60	D
Grado menor que 60	E



Utilizando esta información, escribir un algoritmo que acepte una calificación numérica del estudiante $[0,100]$, convierta esta calificación a su equivalente letra y visualice la calificación correspondiente en letra.

Ejemplo 3 – Solución forma 1



Datos de entrada:

- Nota numérica [0,100]

Datos de salida:

- Nota letra: A, B, C, D, E

Definición de variables:

- Entrada/salida:
 - `nota_num`: Nota numérica
 - `nota_letra`: Nota en letra

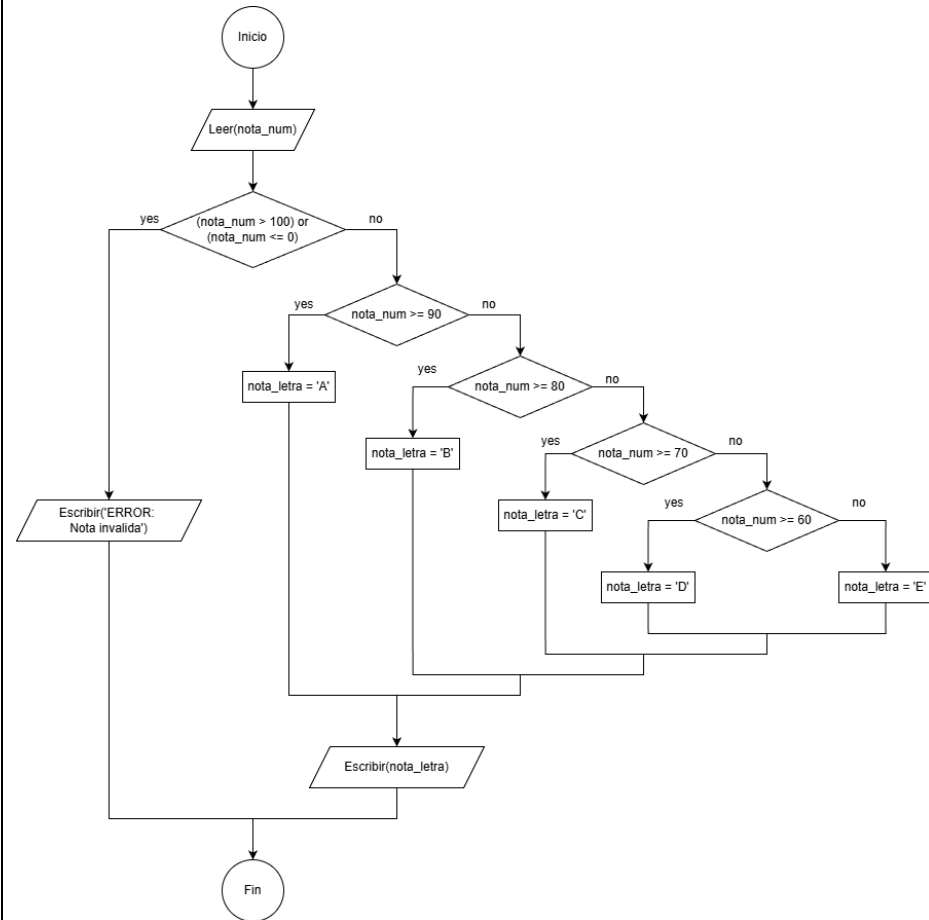
Proceso:

- Verificar el rango al cual pertenece la nota numérica ingresada y de acuerdo a este obtener la correspondiente nota en letras:
 - $90 \leq \text{nota_num} \rightarrow A$
 - $80 \leq \text{nota_num} < 90 \rightarrow B$
 - $70 \leq \text{nota_num} < 80 \rightarrow C$
 - $60 \leq \text{nota_num} < 70 \rightarrow D$
 - $\text{nota_num} < 60 \rightarrow E$
- En caso de que el rango sea invalido (menor que 0 o mayor que 100), colocar un mensaje de error.

Ejemplo 3 – Solución forma 1

Como se puede apreciar en la implementación de pseudocódigo, la indentación es sumamente importante por que permite visualizar los anidamientos de la estructura condicional.

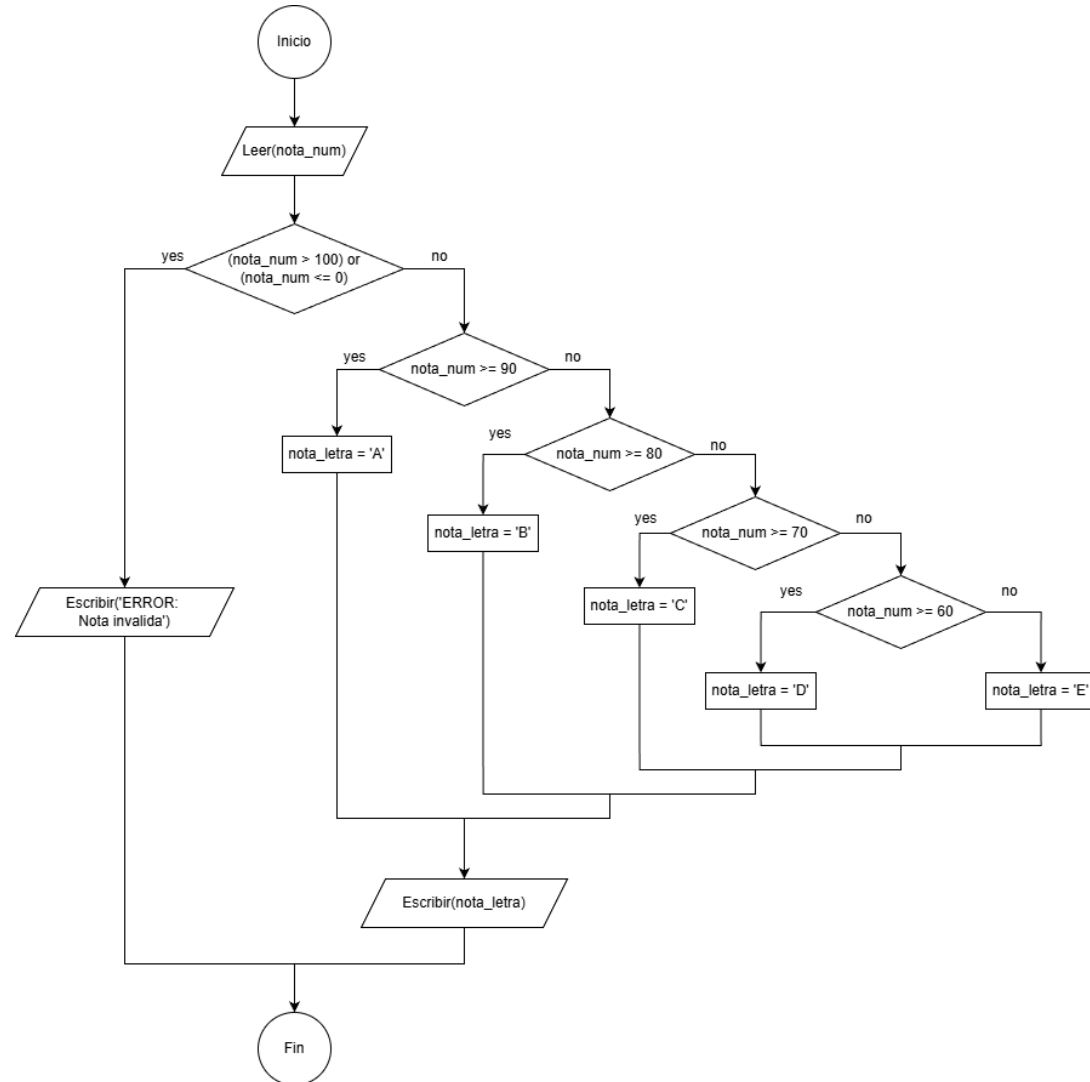
```
Inicio
Leer(nota_num)
Si ((nota_num > 100) or (nota_num < 0)) Entonces
    Escribir('ERROR: Nota invalida')
Sino
    Si (nota_num >= 90) Entonces
        nota_letra = 'A'
    Sino
        Si (nota_num >= 80) Entonces
            nota_letra = 'B'
        Sino
            Si (nota_num >= 70) Entonces
                nota_letra = 'C'
            Sino
                Si (nota_num >= 60) Entonces
                    nota_letra = 'D'
                Sino
                    nota_letra = 'E'
                Fin_Si
            Fin_Si
        Fin_Si
    Fin_Si
    Escribir(nota_letra)
Fin_Si
Fin
```



Ejemplo 3 – Solución forma 1

Como se puede notar, mediante el uso de la estructura condicional se implementa cada una de las posibilidades para la nota numérica ingresada. La siguiente tabla resume estas:

Grado numérico	Grado en letra
$(\text{nota_num} > 100) \text{ or } (\text{nota_num} < 0)$	Nota invalida
$90 \leq \text{nota_num} \leq 100$	A
$80 \leq \text{nota_num} < 90$	B
$70 \leq \text{nota_num} < 80$	C
$60 \leq \text{nota_num} < 70$	D
$\text{nota_num} < 60$	E



Ejemplo 3 – Solución forma 1

Casos de test

notaNum	notaLetra
101	Invalida
80	B
53	E
96	A
67	D
79	C
-1	Invalida

Prueba de escritorio:

notaNum	Salida

```
Inicio
  Leer(nota_num)
  Si ((nota_num > 100) or (nota_num < 0)) Entonces
    Escribir('ERROR: Nota invalida')
  Sino
    Si (nota_num >= 90) Entonces
      nota_letra = 'A'
    Sino
      Si (nota_num >= 80) Entonces
        nota_letra = 'B'
      Sino
        Si (nota_num >= 70) Entonces
          nota_letra = 'C'
        Sino
          Si (nota_num >= 60) Entonces
            nota_letra = 'D'
          Sino
            nota_letra = 'E'
          Fin_Si
        Fin_Si
      Fin_Si
    Fin_Si
  Escribir(nota_letra)
Fin_Si
Fin
```

Alternativa múltiple – **if-else** anidados

- A diferencia del pseudocódigo, Python no tiene un **Fin_Si** que cierre el bloque explícitamente: la **indentación** es la que delimita dónde empieza y termina cada **if/else**. Si la indentación está mal, el programa deja de hacer lo que se espera que haga.

Pseudocódigo

```
Si (condicion_1) Entonces
    condicion_1 verdad
Sino
    Si (condicion_2) Entonces
        condicion_2 verdad
    Sino
        Si (condicion_3) Entonces
            condicion_3 verdad
        Sino
            ...
        Si (condicion_n) Entonces
            condicion_n verdad
        Sino
            condicion_n falso
    Fin_Si
    ...
Fin_Si
Fin_Si
Fin_Si
```

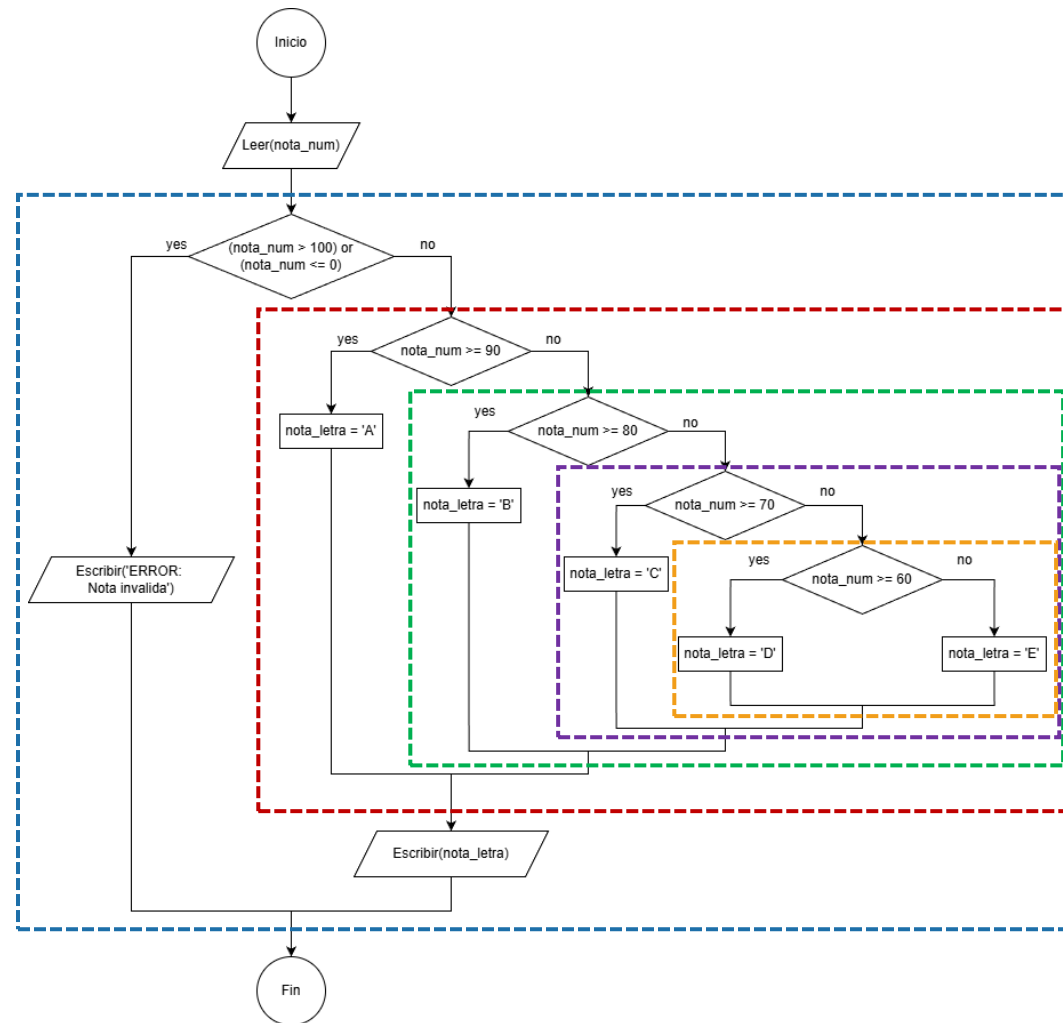
Python

```
if (condicion_1):
    condicion_1 verdad
else:
    if (condicion_2):
        condicion_2 verdad
    else:
        if (condicion_3):
            condicion_3 verdad
        else:
            ...
        if (condicion_n):
            condicion_n verdad
    else:
        condicion_n falso
    ...
```

Ejemplo 3 – Solución en python (forma 1)

- En Python, la indentación es sumamente importante porque indica al interprete el alcance de un bloque (inicio y fin). Indentar mal genera errores de compilación.
- A medida que los anidamientos crecen la indentación tiene que ser más complicada.

```
if ((nota_num > 100) or (nota_num < 0)):  
    print('ERROR: Nota invalida')  
else:  
    if (nota_num >= 90):  
        nota_letra = 'A'  
    else:  
        if (nota_num >= 80):  
            nota_letra = 'B'  
        else:  
            if (nota_num >= 70):  
                nota_letra = 'C'  
            else:  
                if (nota_num >= 60):  
                    nota_letra = 'D'  
                else:  
                    nota_letra = 'E'  
            print(f'{nota_num}: {nota_letra}')
```



Alternativa múltiple – **if-elif-else**

Python ofrece la palabra clave **elif** (contracción de "else if") para evitar que la indentación crezca con cada nueva condición: en vez de anidar un if dentro del else anterior, cada alternativa queda al mismo nivel de indentación.

Pseudocódigo

```
Si (condicion_1) Entonces
    condicion_1 verdad
Sino
    Si (condicion_2) Entonces
        condicion_2 verdad
    Sino
        Si (condicion_3) Entonces
            condicion_3 verdad
        Sino
            ...
            Si (condicion_n) Entonces
                condicion_n verdad
            Sino
                condicion_n falso
        Fin_Si
    ...
    Fin_Si
Fin_Si
```

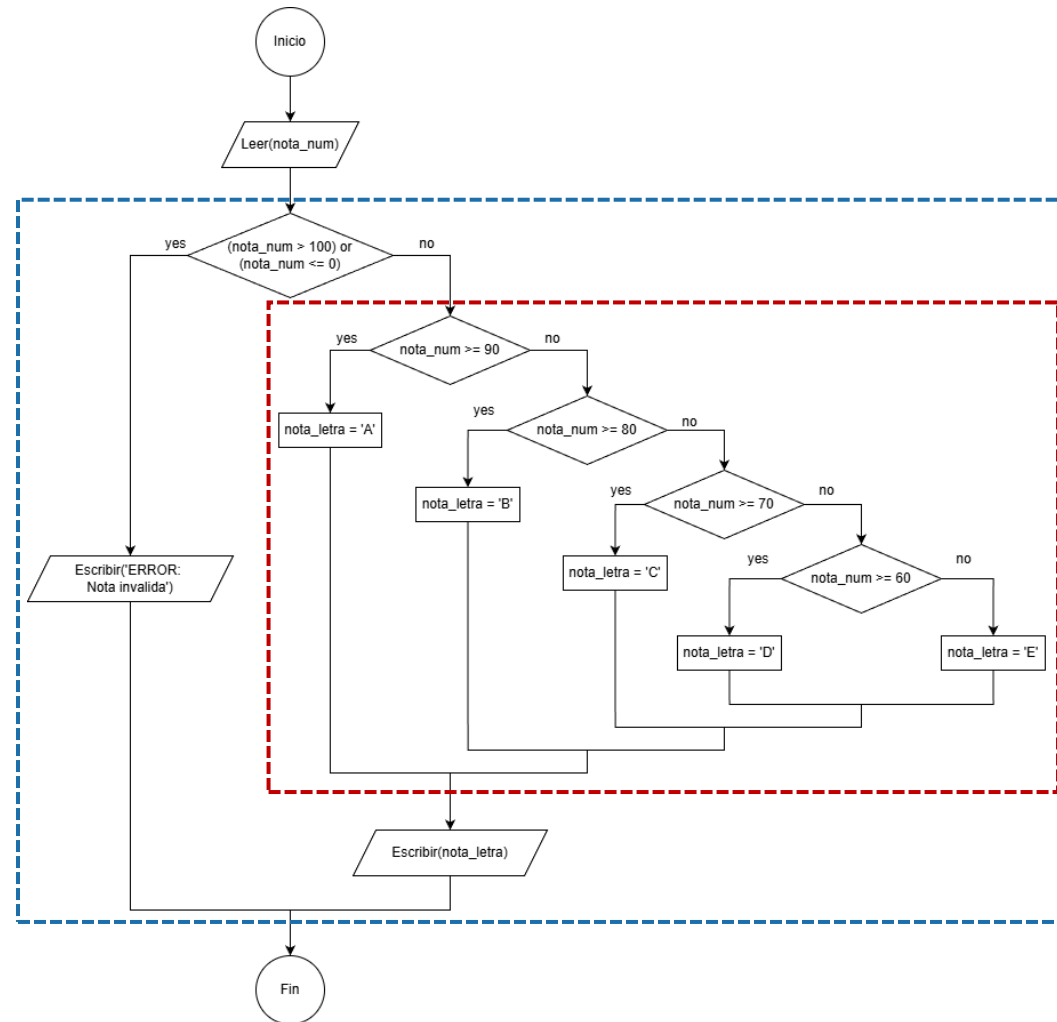
Python

```
if (condicion_1):
    condicion_1 verdad
elif (condicion_2):
    condicion_2 verdad
elif (condicion_3):
    condicion_3 verdad
...
elif (condicion_n):
    condicion_n verdad
else:
    condicion_n falso
```

Ejemplo 3 – Solución en python (forma 2)

- Dentro del recuadro rojo, todas las condiciones evalúan el mismo criterio (rangos de **nota_num**), por eso se pueden encadenar con **elif**. El recuadro azul es una alternativa doble aparte — separa el caso de nota inválida del resto — y no forma parte de esa cadena.

```
if ((nota_num > 100) or (nota_num < 0)):
    print('ERROR: Nota invalida')
else:
    if (nota_num >= 90):
        nota_letra = 'A'
    elif (nota_num >= 80):
        nota_letra = 'B'
    elif (nota_num >= 70):
        nota_letra = 'C'
    elif (nota_num >= 60):
        nota_letra = 'D'
    else:
        nota_letra = 'E'
    print(f'{nota_num}: {nota_letra}')
```



Ejemplo 3 – Comparación implementaciones

- Cuando todas las condiciones evalúan el mismo criterio, usar **elif** reduce el anidamiento y facilita la lectura del código.
- Ambas versiones son funcionalmente equivalentes: mismo resultado, distinta forma.
- Aplique solo si el criterio es el mismo; con criterios distintos sigue siendo necesario anidar **if**.

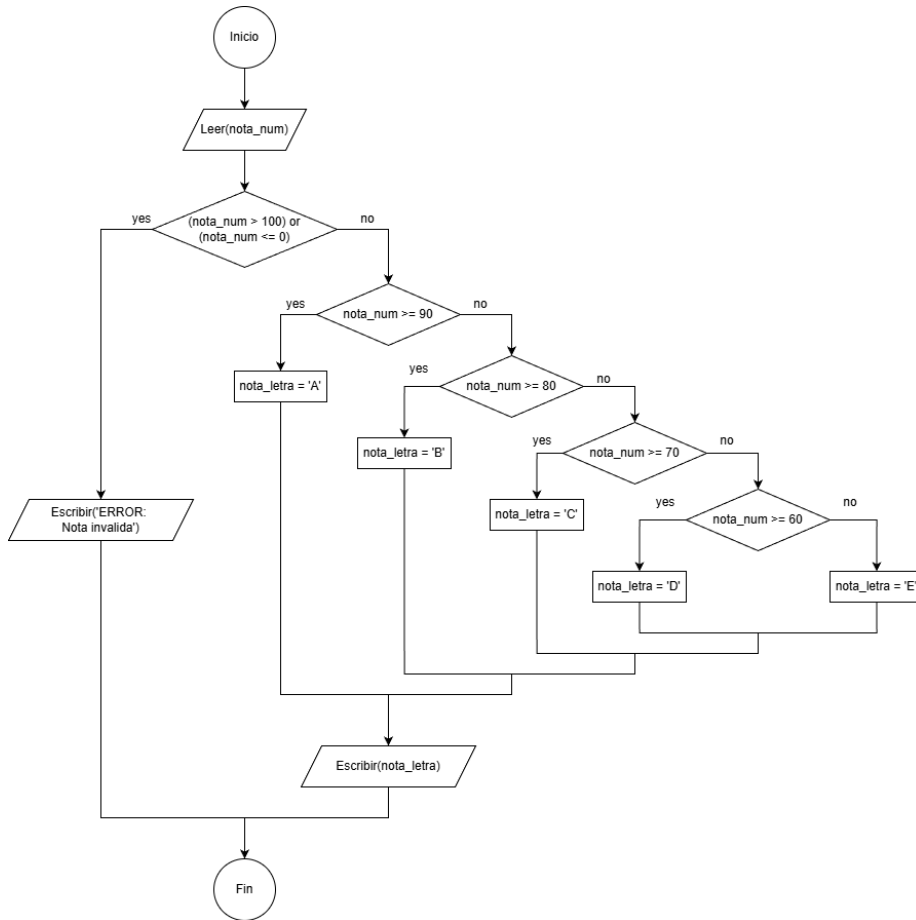
```
if ((nota_num > 100) or (nota_num < 0)):
    print('ERROR: Nota invalida')
else:
    if (nota_num >= 90):
        nota_letra = 'A'
    else:
        if (nota_num >= 80):
            nota_letra = 'B'
        else:
            if (nota_num >= 70):
                nota_letra = 'C'
            else:
                if (nota_num >= 60):
                    nota_letra = 'D'
                else:
                    nota_letra = 'E'
    print(f'{nota_num}: {nota_letra}')
```

Comparación



```
if ((nota_num > 100) or (nota_num < 0)):
    print('ERROR: Nota invalida')
else:
    if (nota_num >= 90):
        nota_letra = 'A'
    elif (nota_num >= 80):
        nota_letra = 'B'
    elif (nota_num >= 70):
        nota_letra = 'C'
    elif (nota_num >= 60):
        nota_letra = 'D'
    else:
        nota_letra = 'E'
    print(f'{nota_num}: {nota_letra}')
```

Ejemplo 3 – Resumen solución forma 1



Inicio

Leer(nota_num)

Si ((nota_num > 100) or (nota_num < 0)) Entonces
Escribir('ERROR: Nota invalida')

Sino

Si (nota_num >= 90) Entonces
nota_letra = 'A'

Sino

Si (nota_num >= 80) Entonces
nota_letra = 'B'

Sino

Si (nota_num >= 70) Entonces
nota_letra = 'C'

Sino

Si (nota_num >= 60) Entonces
nota_letra = 'D'

Sino

nota_letra = 'E'

Fin_Si

Fin_Si

Fin_Si

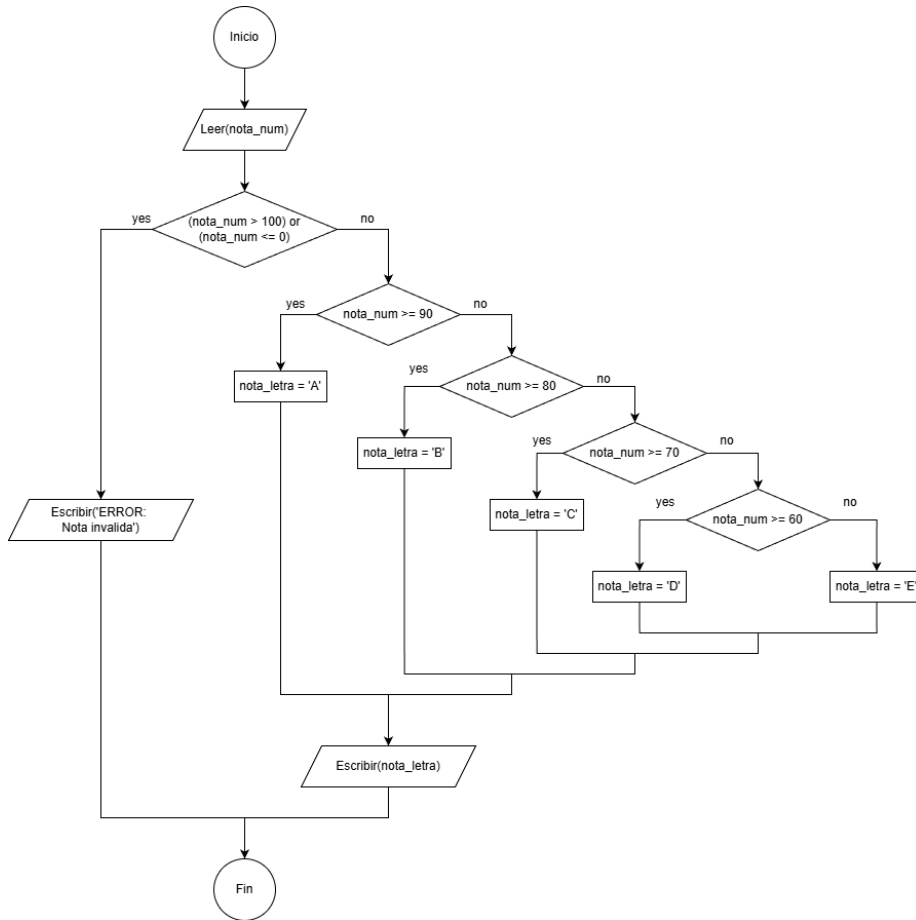
Fin_Si

Escribir(nota_letra)

Fin_Si

Fin

Ejemplo 3 – Resumen solución forma 1



Inicio

Leer(nota_num)

Si ((nota_num > 100) or (nota_num < 0)) Entonces
Escribir('ERROR: Nota invalida')

Sino

Si (nota_num >= 90) Entonces
nota_letra = 'A'

Sino

Si (nota_num >= 80) Entonces
nota_letra = 'B'

Sino

Si (nota_num >= 70) Entonces
nota_letra = 'C'

Sino

Si (nota_num >= 60) Entonces
nota_letra = 'D'

Sino

nota_letra = 'E'

Fin_Si

Fin_Si

Fin_Si

Fin_Si

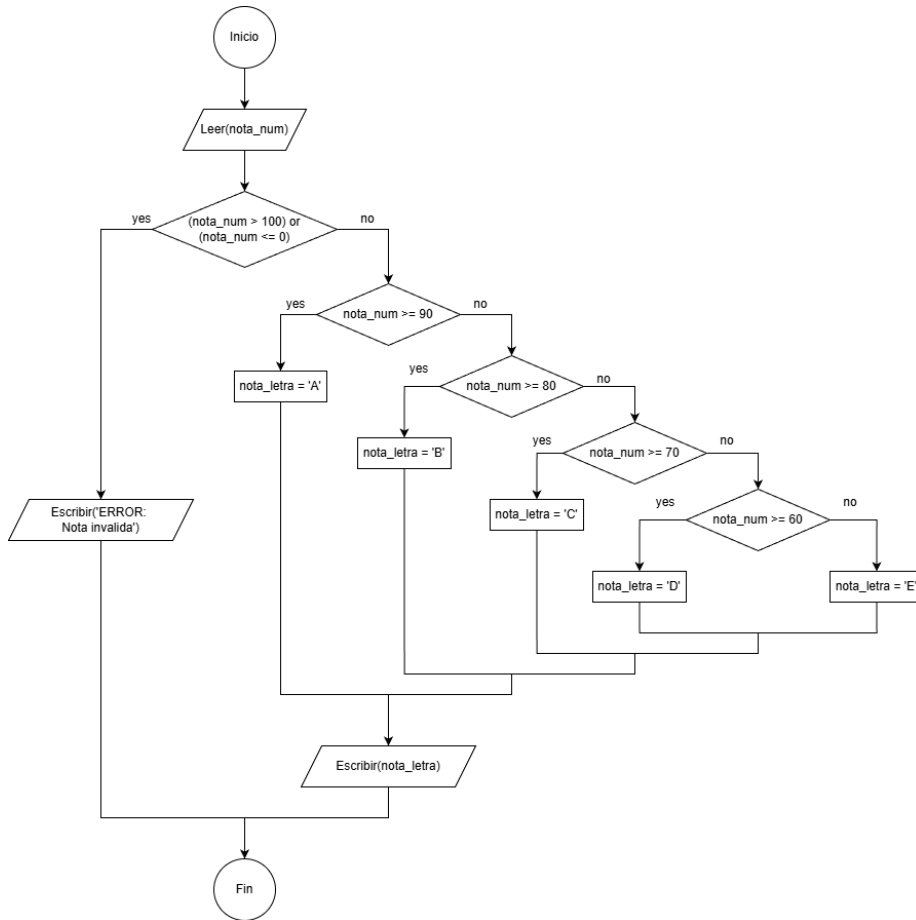
Escribir(nota_letra)

Fin_Si

Fin

Ejemplo 3 – Resumen solución forma 1

Implementación usando estructuras if-else anidados

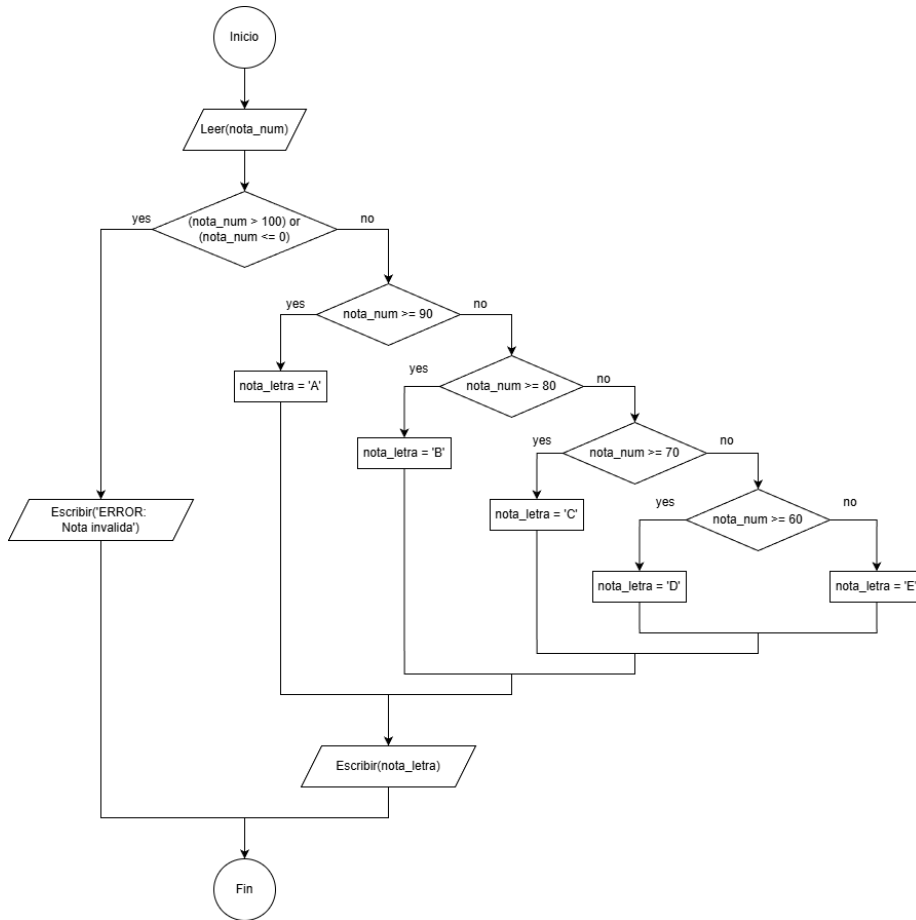


```
nota_num = int(input("Digite la nota [0,100]: "))
if ((nota_num > 100) or (nota_num < 0)):
    print('ERROR: Nota invalida')
else:
    if (nota_num >= 90):
        nota_letra = 'A'
    else:
        if (nota_num >= 80):
            nota_letra = 'B'
        else:
            if (nota_num >= 70):
                nota_letra = 'C'
            else:
                if (nota_num >= 60):
                    nota_letra = 'D'
                else:
                    nota_letra = 'E'
    print(f'{nota_num}: {nota_letra}')
```

[Link de ejecución](#)

Ejemplo 3 – Resumen solución forma 1

Implementación usando estructuras if-elif-else



```
nota_num = int(input("Digite la nota [0,100]: "))
if ((nota_num > 100) or (nota_num < 0)):
    print('ERROR: Nota invalida')
else:
    if (nota_num >= 90):
        nota_letra = 'A'
    elif (nota_num >= 80):
        nota_letra = 'B'
    elif (nota_num >= 70):
        nota_letra = 'C'
    elif (nota_num >= 60):
        nota_letra = 'D'
    else:
        nota_letra = 'E'
    print(f'{nota_num}: {nota_letra}')
```

[Link de ejecución](#)

Ejemplo 3 – Solución forma 2

Esta manera también cumple los requisitos del problema. Simplemente cambia la forma como se usan los condicionales.



Datos de entrada:

- Nota numérica [0,100]

Datos de salida:

- Nota letra: A, B, C, D, E

Definición de variables:

- Entrada/salida:
 - `nota_num`: Nota numérica
 - `nota_letra`: Nota en letra

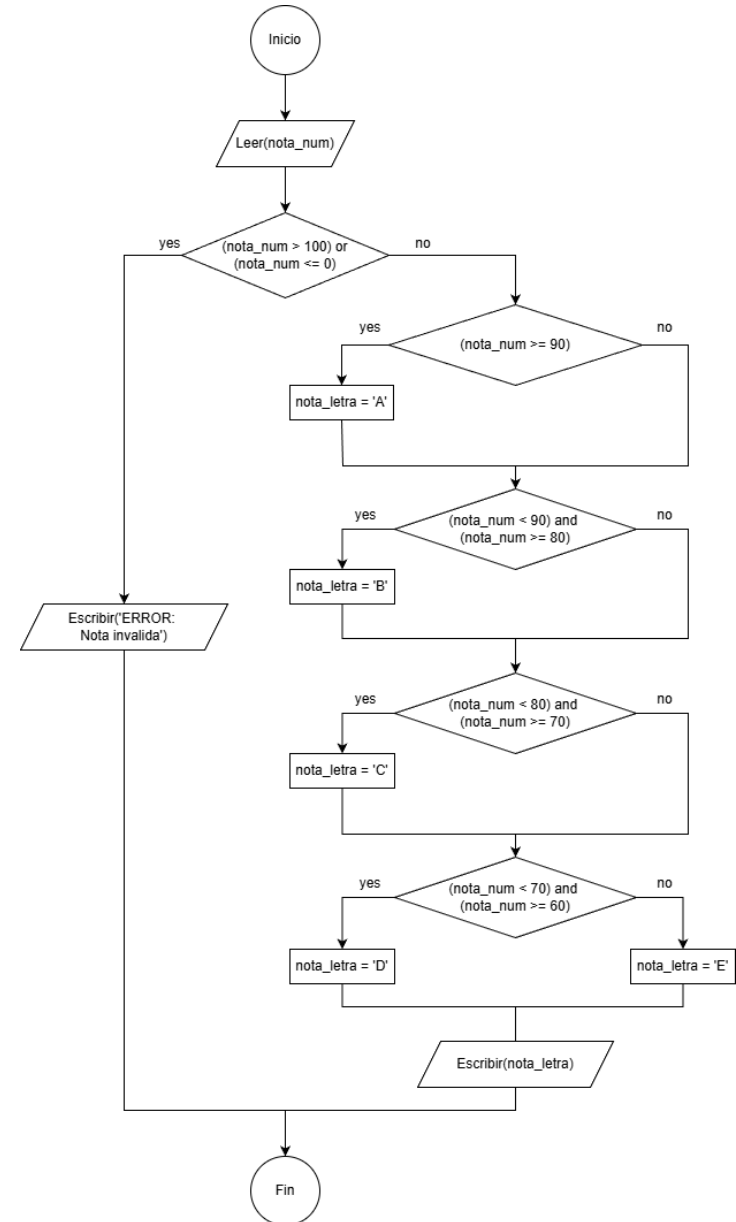
Proceso:

- Verificar el rango al cual pertenece la nota numérica ingresada y de acuerdo a este obtener la correspondiente nota en letras:
 - $90 \leq \text{nota_num} \rightarrow A$
 - $80 \leq \text{nota_num} < 90 \rightarrow B$
 - $70 \leq \text{nota_num} < 80 \rightarrow C$
 - $60 \leq \text{nota_num} < 70 \rightarrow D$
 - $\text{nota_num} < 60 \rightarrow E$
- En caso de que el rango sea invalido (menor que 0 o mayor que 100), colocar un mensaje de error.

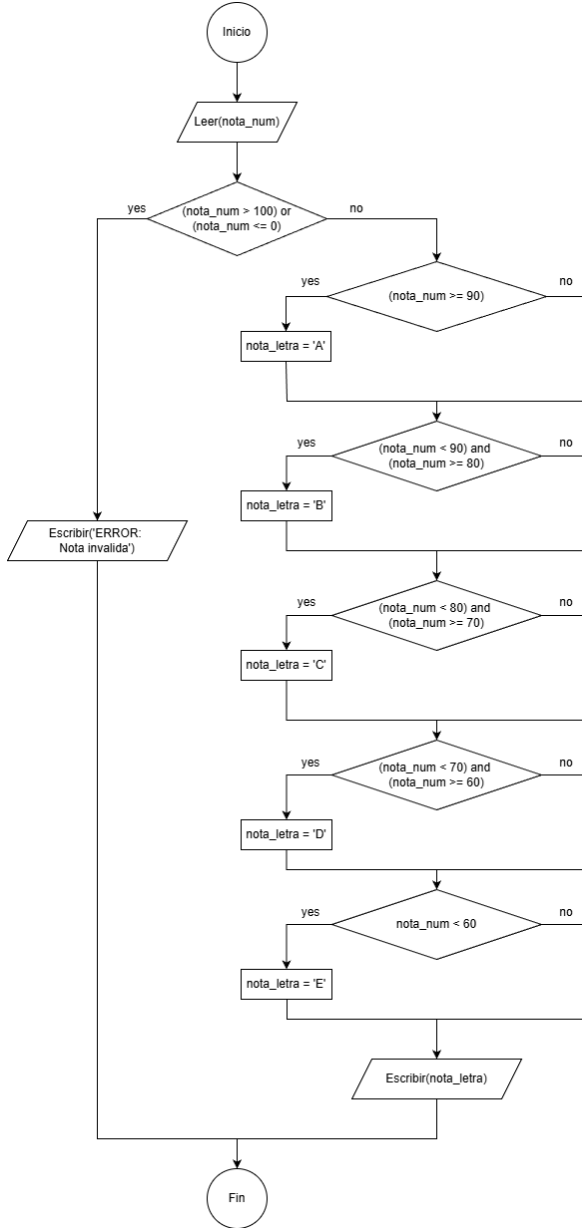
Ejemplo 3 – Solución forma 2

La siguiente estructura del algoritmo sigue siendo fiel a la tabla, sin embargo la implementación lógica es diferente a la anteriormente planteada:

Grado numérico	Grado en letra
$(\text{nota_num} > 100) \text{ or } (\text{nota_num} < 0)$	Nota invalida
$90 \leq \text{nota_num} \leq 100$	A
$80 \leq \text{nota_num} < 90$	B
$70 \leq \text{nota_num} < 80$	C
$60 \leq \text{nota_num} < 70$	D
$\text{nota_num} < 60$	E



Ejemplo 3 – Solución forma 2



Inicio

Leer(nota_num)

Si ((nota_num > 100) or (nota_num < 0)) Entonces
Escribir('ERROR: Nota invalida')

Sino

Si (nota_num >= 90) Entonces
nota_letra = 'A'

Fin_Si

Si ((nota_num < 90) and (nota_num >= 80)) Entonces
nota_letra = 'B'

Fin_Si

Si ((nota_num < 80) and (nota_num >= 70)) Entonces
nota_letra = 'C'

Fin_Si

Si ((nota_num < 70) and (nota_num >= 60)) Entonces
nota_letra = 'D'

Fin_Si

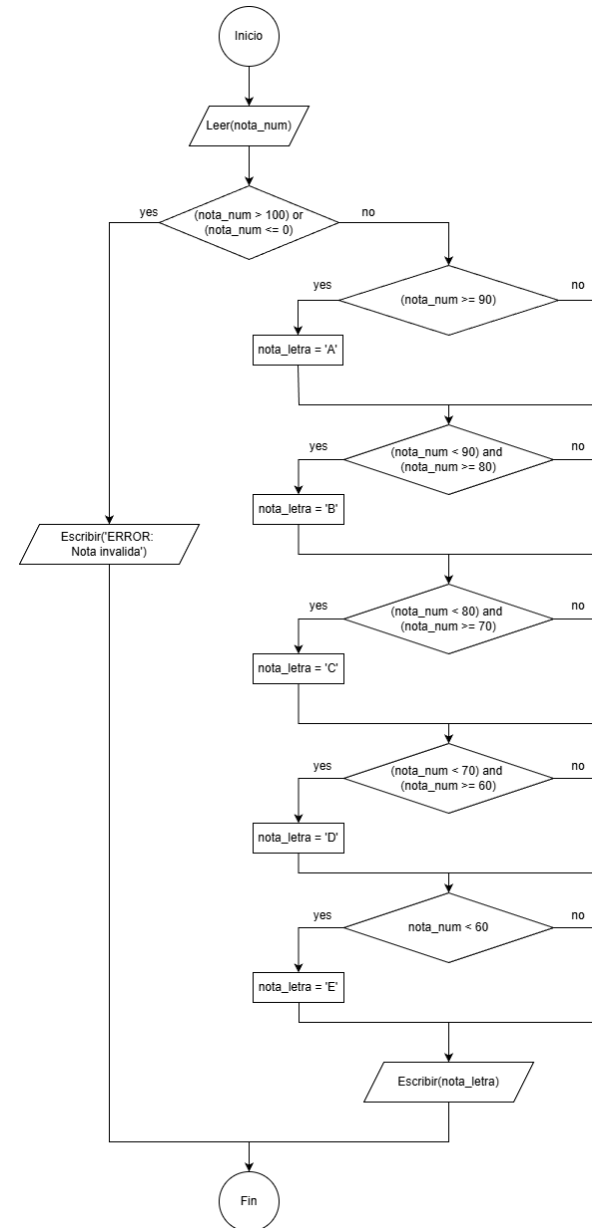
Si (nota_num < 60) Entonces
nota_letra = 'E'

Fin_Si

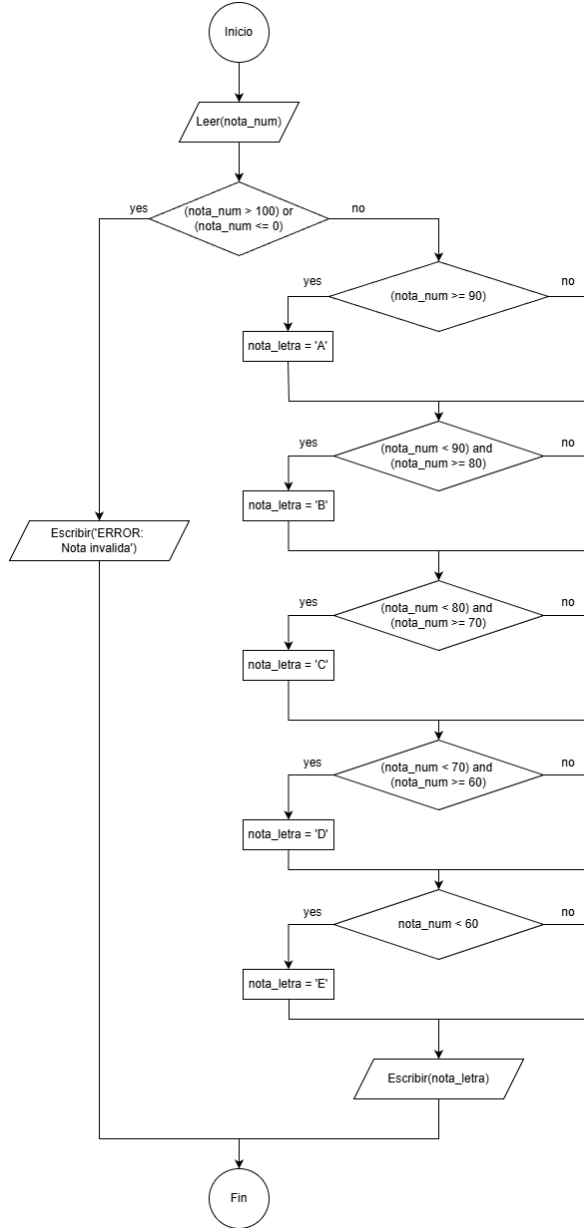
Escribir(nota_letra)

Fin_Si

Fin



Ejemplo 3 – Codificación en python (forma 1)

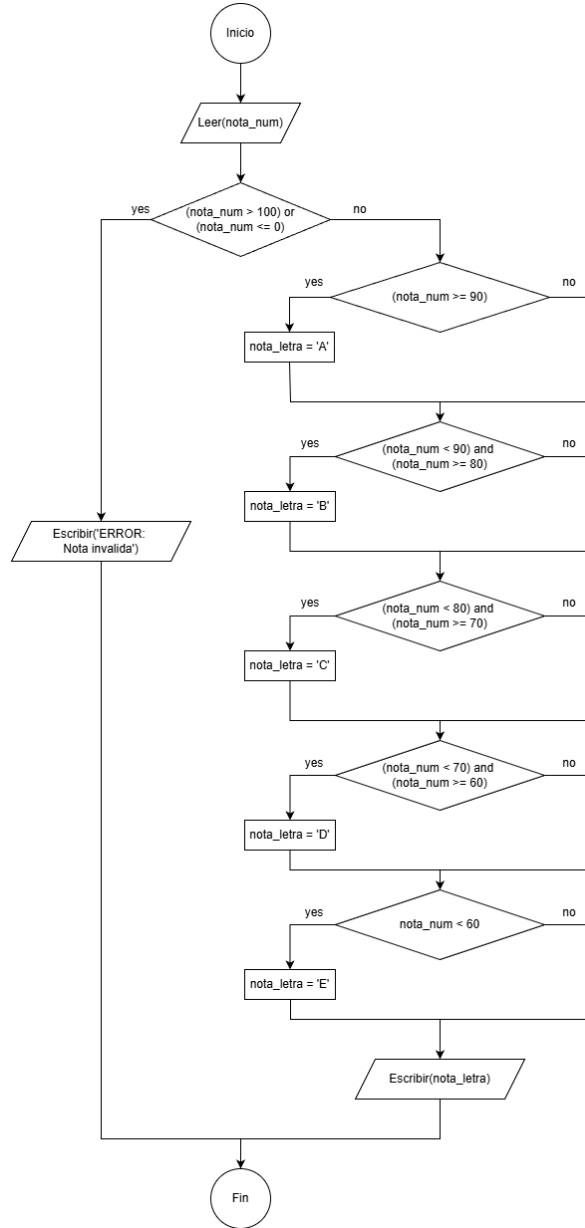


En este caso, se implementa cada condición empleando operadores lógicos de manera explícita para expresar los rangos.

```
nota_num = int(input("Digite la nota [0,100]: "))
if ((nota_num > 100) or (nota_num < 0)):
    print('ERROR: Nota invalida')
else:
    if (nota_num >= 90):
        nota_letra = 'A'
    if ((nota_num < 90) and (nota_num >= 80)):
        nota_letra = 'B'
    if ((nota_num < 80) and (nota_num >= 70)):
        nota_letra = 'C'
    if ((nota_num < 70) and (nota_num >= 60)):
        nota_letra = 'D'
    if (nota_num < 60):
        nota_letra = 'E'
    print(f'{nota_num}: {nota_letra}')
```

[Link de ejecución](#)

Ejemplo 3 – Codificación en python (forma 2)



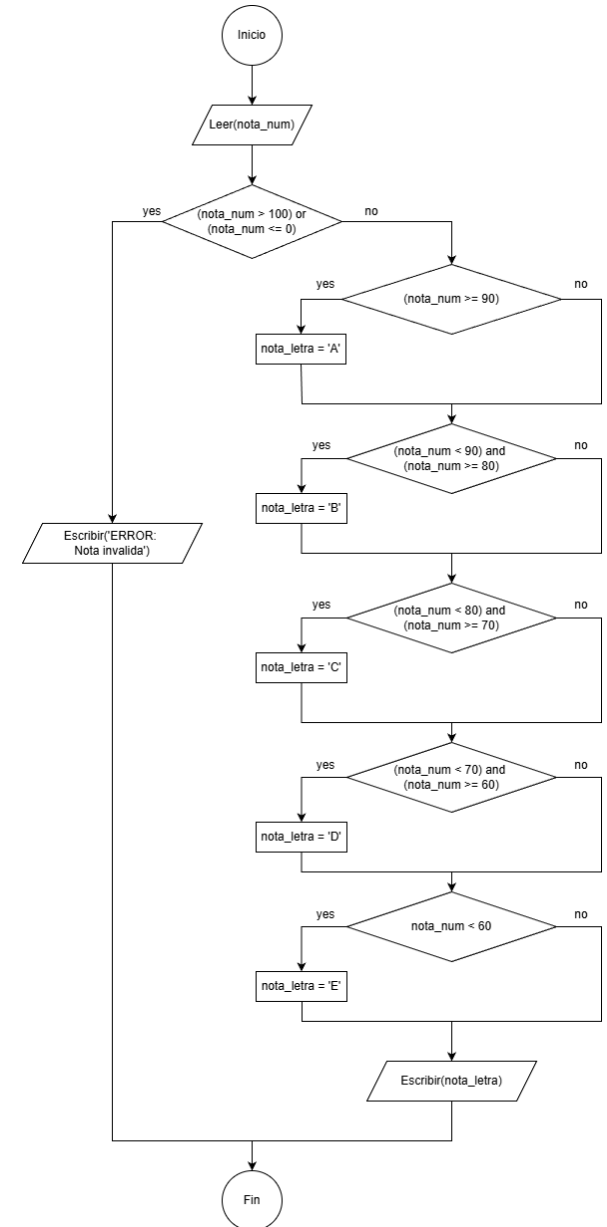
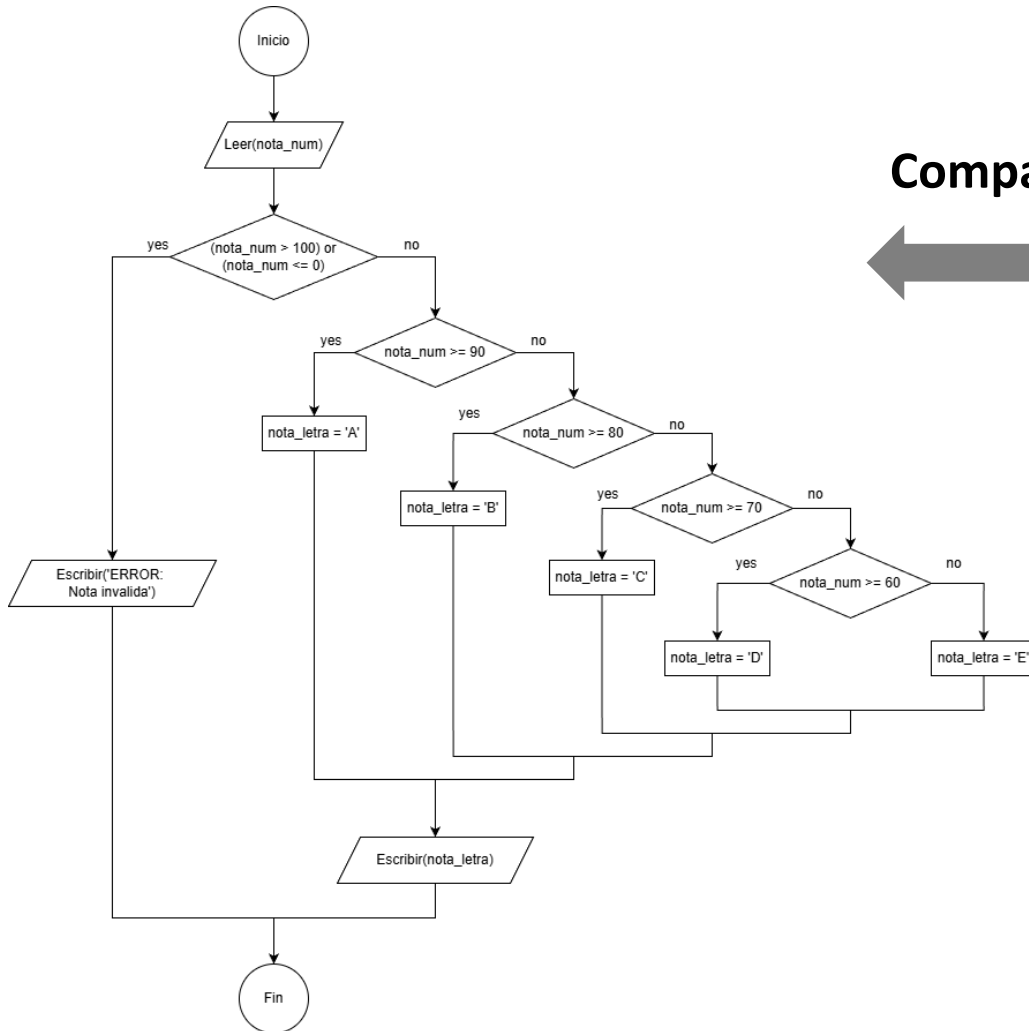
Teniendo en cuenta que Python soporta encadenamiento de operadores, los rangos empleados para las condiciones se expresan de manera similar a como se haría matemáticamente.

```
nota_num = int(input("Digite la nota [0,100]: "))
if ((nota_num > 100) or (nota_num < 0)):
    print('ERROR: Nota invalida')
else:
    if (nota_num >= 90):
        nota_letra = 'A'
    if (80 <= nota_num < 90):
        nota_letra = 'B'
    if (70 <= nota_num < 80):
        nota_letra = 'C'
    if (60 <= nota_num < 70):
        nota_letra = 'D'
    if (nota_num < 60):
        nota_letra = 'E'
    print(f'{nota_num}: {nota_letra}')
```

[Link de ejecución](#)

Ejemplo 3 – Comparación de soluciones

Comparación



Ejemplo 3 – Comparación de soluciones

- Existen múltiples formas válidas de implementar la misma lógica; lo esencial es que el control de flujo produzca el mismo comportamiento.
- Determinar cuál implementación es "mejor" (más eficiente, más compacta) corresponde al **análisis de algoritmos**, un tema fuera del alcance de este curso.
- Lo más importante es la creatividad para traducir una idea en instrucciones ejecutables por una máquina: la lógica define el algoritmo (independiente del lenguaje), y el lenguaje de programación es el medio para expresarlo de forma que la máquina lo entienda.

Ejemplos de repaso

4. Se desea obtener la nomina semanal (salario neto) – de los empleados de una empresa cuyo trabajo se paga por horas y del siguiente modo:
- Las horas inferiores o iguales a 35 horas (normales) se pagan a una tarifa que se debe introducir por teclado igual que el numero de horas y el nombre del trabajador.
 - Las horas superiores a 35 se pagaran como extra a un precio de 1.5 horas normales.
 - Los impuestos a deducir a los trabajadores varían en función de su sueldo mensual:
 - Si sueldo $\leq$ 300000, libre de impuestos.
 - Los siguientes 150000 al 20%.
 - El resto al 30%.

Ejemplo 4 - Solución

Se desea obtener la **nomina semanal (salario neto)** – de los empleados de una empresa cuyo trabajo se paga por **horas** y del siguiente modo:

- Las horas inferiores o iguales a 35 horas (normales) se pagan a una **tarifa** que se debe introducir por teclado igual que el numero de horas y el **nombre del trabajador**.
- Las horas superiores a 35 se pagaran como extra a un precio de 1.5 horas normales.
- Los **impuestos** a deducir a los trabajadores varían en función de su sueldo neto:
 - Si sueldo ≤ 300000 , libre de impuestos.
 - Los siguientes 150000 al 20%.
 - El resto al 30%.

Ejemplo 4 - Solución

Datos de entrada:

- Nombre.
- Horas trabajadas.
- Tarifa hora.

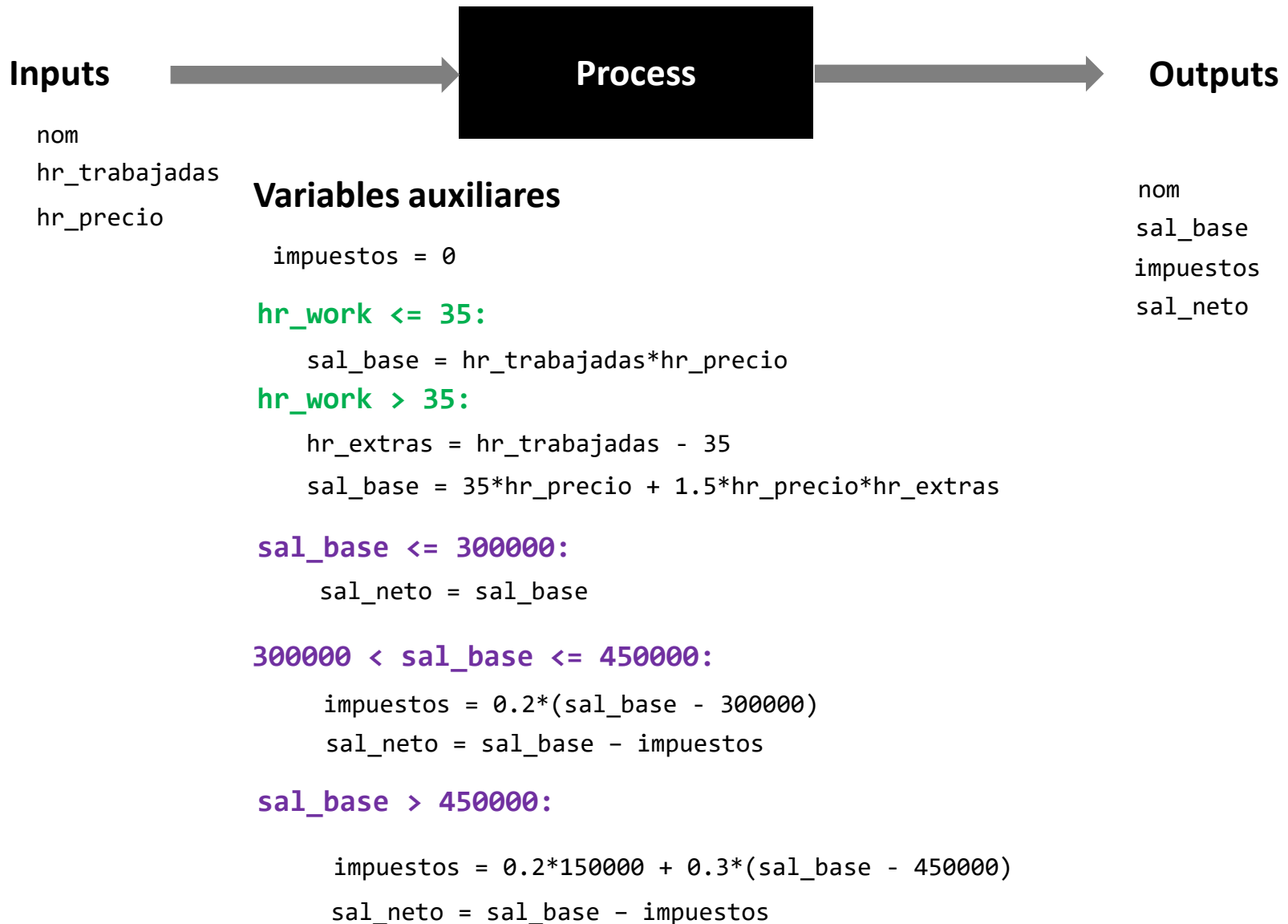
Datos de salida:

- Salario neto.
- Nombre
- Impuestos.
- Salario bruto.

Proceso:

1. Solicitar datos de entrada (nombre, valor hora, tarifa por hora).
2. Verificar horas trabajadas
3. Calcular impuestos
4. Calculo del salario neto
5. Mostrar datos de salida (nombre, salario base, impuestos, salario neto)

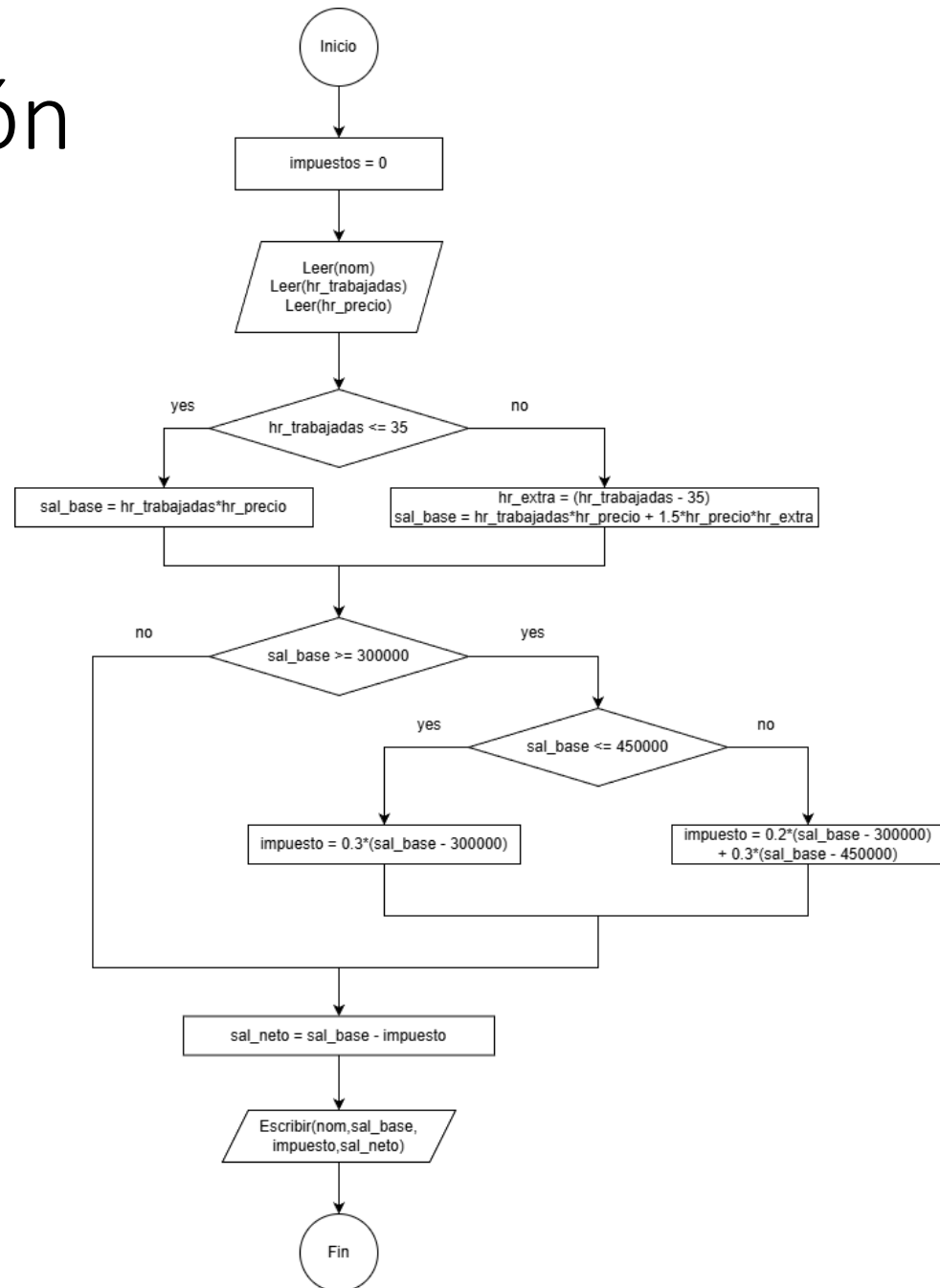
Ejemplo 4 - Solución



Ejemplo 4 - Solución

Proceso:

1. Solicitar datos de entrada (nombre, valor hora, tarifa por hora).
2. Verificar horas trabajadas
3. Calcular impuestos
4. Calculo del salario neto
5. Mostrar datos de salida (nombre, salario base, impuestos, salario neto)



Ejemplo 4 - Solución

Inicio

impuesto = 0

Leer(nom)

Leer(hr_trabajadas)

Leer(hr_precio)

Si (hr_trabajadas <= 35) Entonces

sal_base = hr_trabajadas*hr_precio

Sino

hr_extra = hr_trabajadas - 35

sal_base = 35*hr_precio + \

1.5*hr_precio*hr_extra

Fin_Si

Si (sal_base >= 300000) Entonces

Si (sal_base <= 450000) Entonces

impuesto = 0.2*(sal_base - 300000)

Sino

impuesto = 0.2*150000 + \

0.3*(sal_base - 450000)

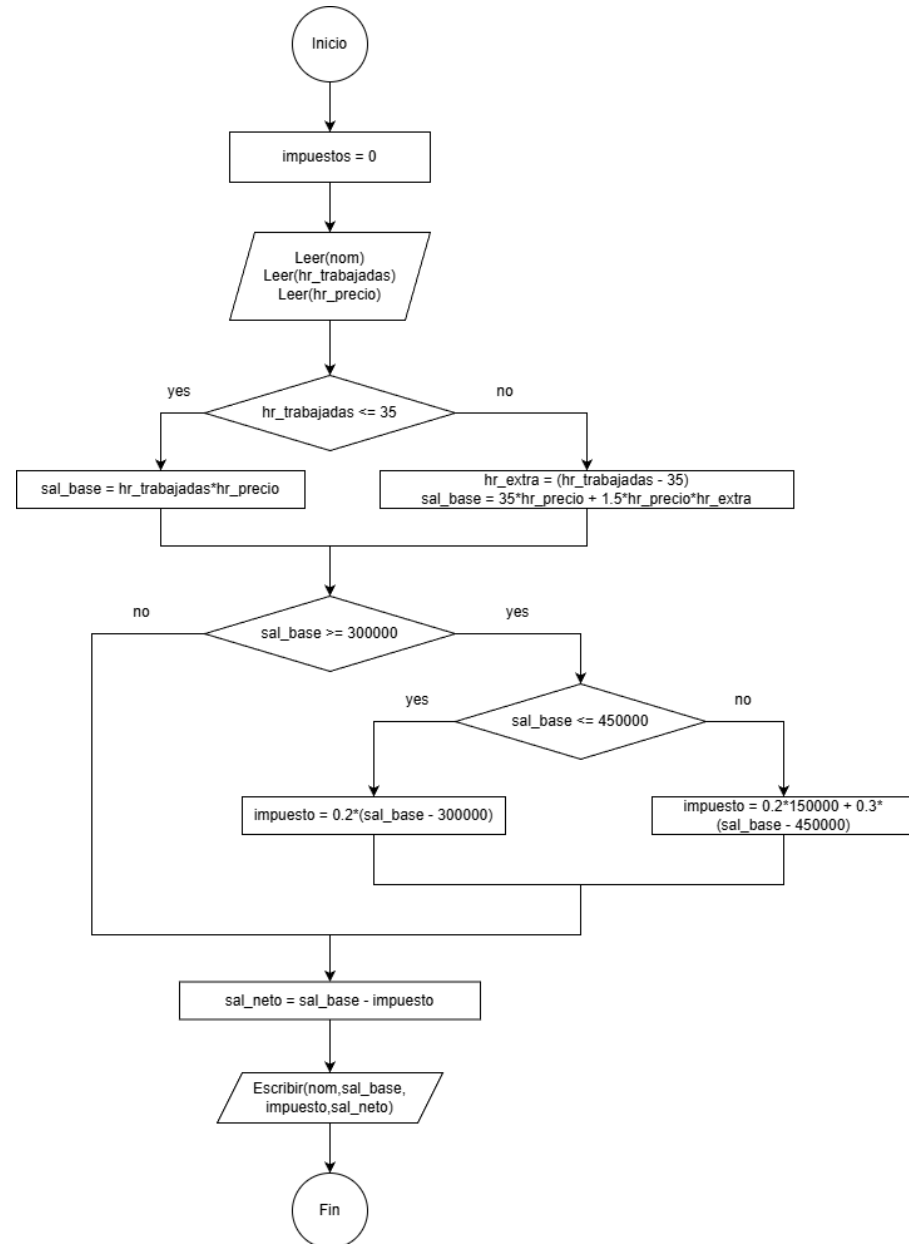
Fin_Si

Fin_Si

sal_neto = sal_base - impuesto

Escribir(nom,sal_base,impuesto,sal_neto)

Fin



Ejemplo 4 - Solución

```
Inicio
    impuesto = 0
    Leer(nom)
    Leer(hr_trabajadas)
    Leer(hr_precio)
    Si (hr_trabajadas <= 35) Entonces
        sal_base = hr_trabajadas*hr_precio
    Sino
        hr_extra = hr_trabajadas - 35
        sal_base = 35*hr_precio + \
            1.5*hr_precio*hr_extra
    Fin_Si
    Si (sal_base >= 300000) Entonces
        Si (sal_base <= 450000) Entonces
            impuesto = 0.2*(sal_base - 300000)
        Sino
            impuesto = 0.2*150000 + \
                0.3*(sal_base - 450000)
        Fin_Si
    Fin_Si
    sal_netto = sal_base - impuesto
    Escribir(nom,sal_base,impuesto,sal_netto)
Fin
```

```
HORAS_NORMALES = 35
TARIFA_EXTRA = 1.5
impuesto = 0

nom = input("Digite el nombre del trabajador: ")
hr_trabajadas = float(input("Digite las horas trabajadas: "))
hr_precio = float(input("Digite la tarifa por hora: "))

if (hr_trabajadas <= HORAS_NORMALES):
    sal_base = hr_trabajadas*hr_precio
else:
    hr_extra = hr_trabajadas - HORAS_NORMALES
    sal_base = HORAS_NORMALES*hr_precio + \
        TARIFA_EXTRA*hr_precio*hr_extra

if (sal_base >= 300000):
    if (sal_base <= 450000):
        impuesto = 0.2 * (sal_base - 300000)
    else:
        impuesto = 0.2*150000 + 0.3*(sal_base - 450000)

sal_netto = sal_base - impuesto

print(f'{nom}, salario base: {sal_base}, \
    impuesto: {impuesto}, salario neto: {sal_netto}')
```

[Link de ejecución](#)

Ejemplo 4 - Solución

Casos de test

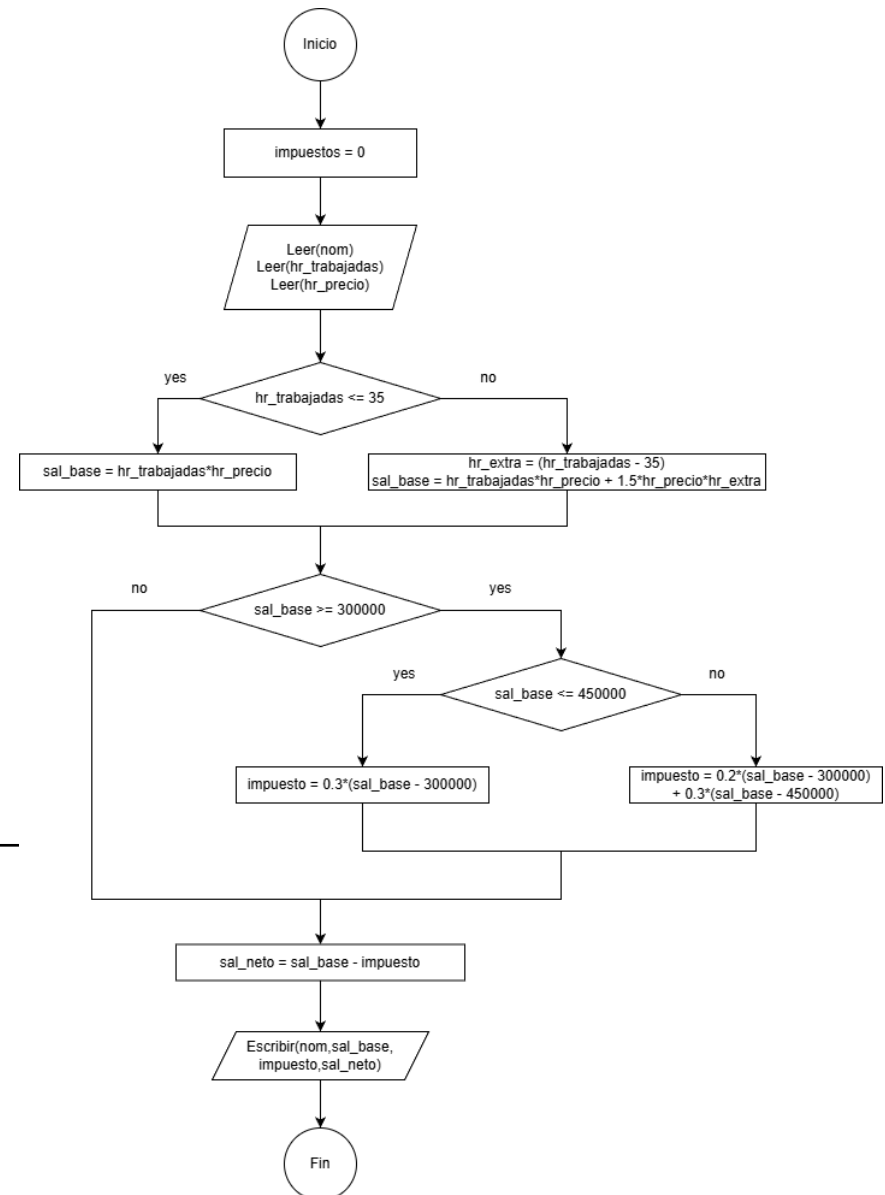
horas	Valor hora	Salario base	impuestos	Salario neto
30	10000	300000	0	300000
40	10000	425000	25000	400000
45	15000	750000	120000	630000



Horas	30
Valor hora	10000
Salario bruto	300000
Impuestos	0
Salario neto	300000

Prueba de escritorio:

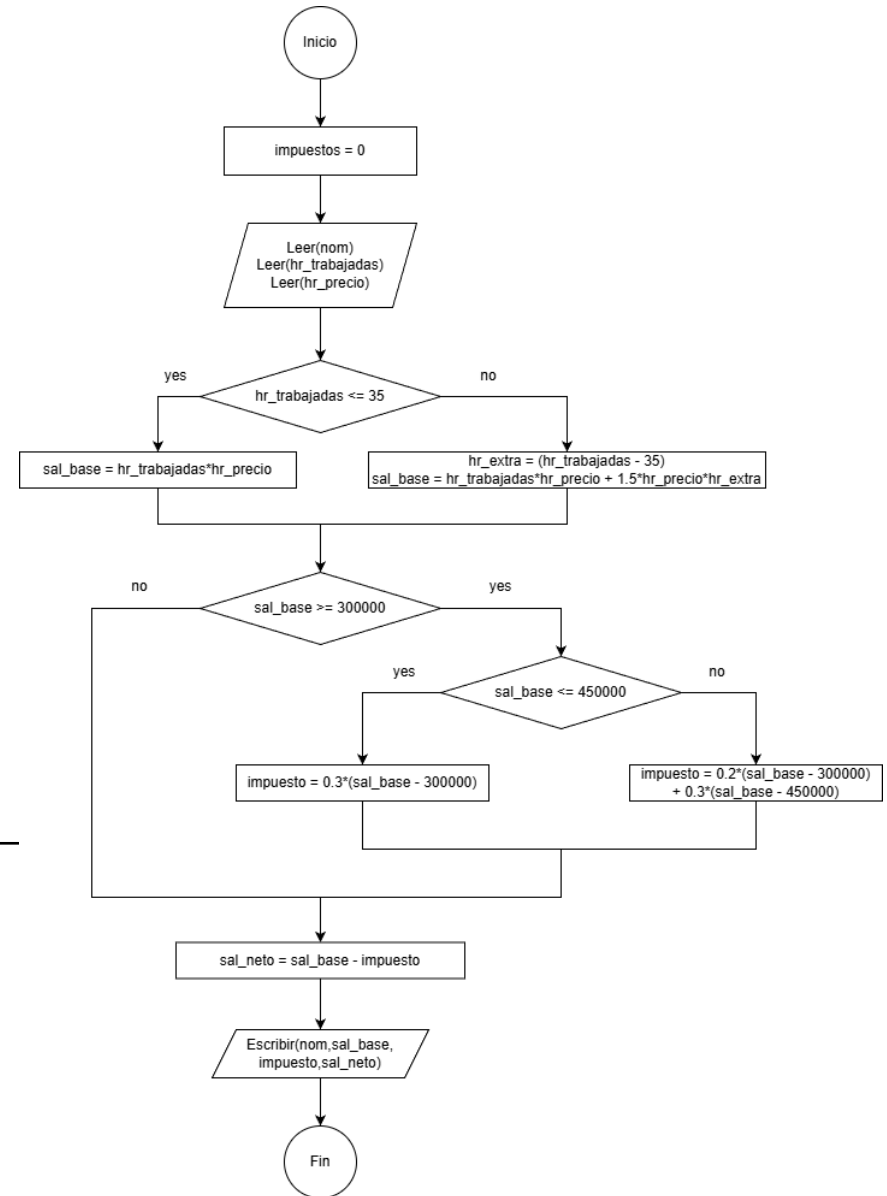
horas	Valor hora	Salario base	Impuestos	Salario neto



Horas	40
Valor hora	10000
Salario bruto	425000
Impuestos	25000
Salario neto	400000

Prueba de escritorio:

horas	Valor hora	Salario base	Impuestos	Salario neto

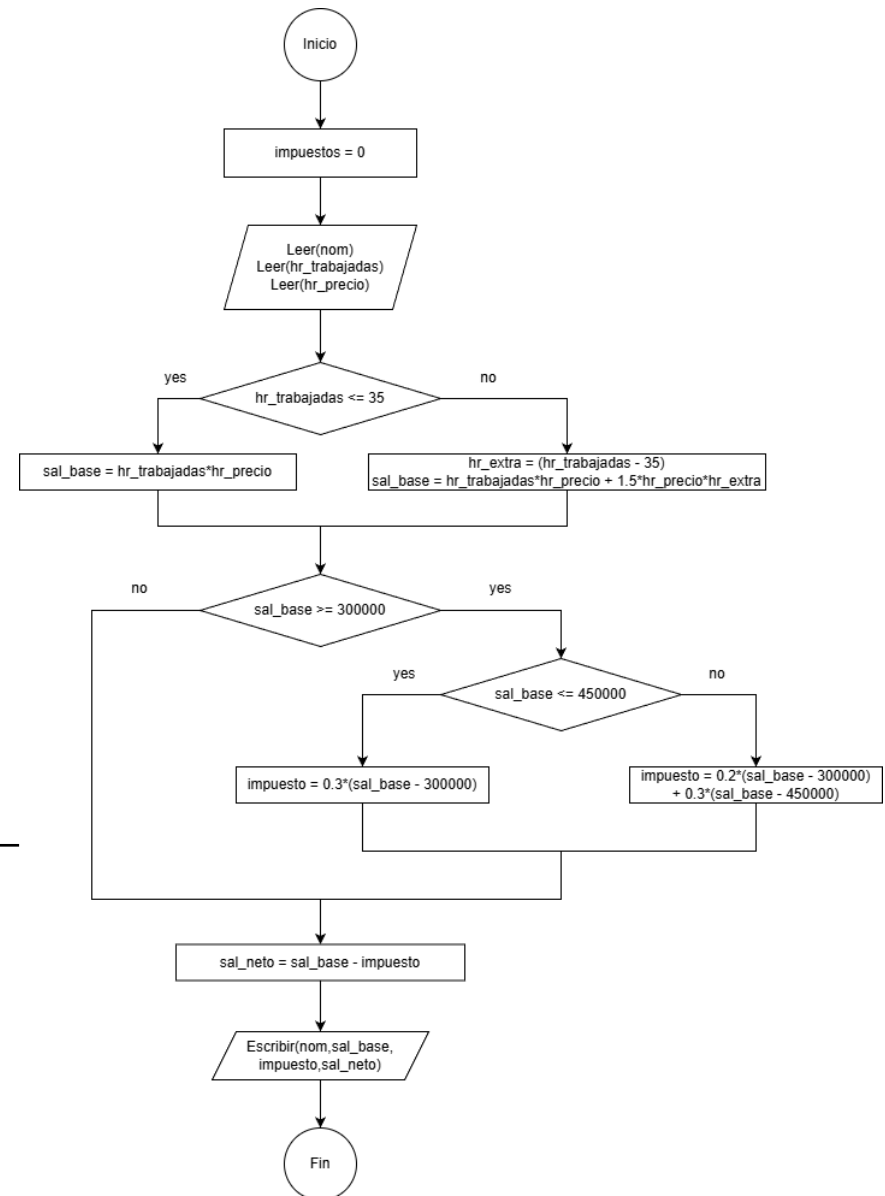


Link de ejecución

Horas	45
Valor hora	15000
Salario bruto	750000
Impuestos	120000
Salario neto	630000

Prueba de escritorio:

horas	Valor hora	Salario base	Impuestos	Salario neto



Ejercicios de repaso

1. Una frutería ofrece las manzanas con descuento según la siguiente tabla:

# de manzanas compradas	% de descuento
0 – 2	0%
3 – 5	10%
6 – 10	15%
11 en adelante	20%

Determinar cuanto pagara una persona que compre manzanas en esa frutería, dado el precio por unidad.

2. Se tienen cuatro esferas (A, B, C, D) de las cuales se sabe que tres son de igual peso y una diferente. Elaborar un algoritmo que determine cuál es la esfera diferente.

Ejercicios de repaso

3. Diseñar un algoritmo que lea tres números A, B y C y visualice en pantalla el valor mas grande. Se supone que los 3 valores son diferentes.
4. Escribir un procedimiento para leer los valores de dos de los tres ángulos internos de un triángulo y mostrar en pantalla “Es un Triángulo Rectángulo” si efectivamente es un triángulo de este tipo o, en caso contrario, mostrar “No es un Triángulo Rectángulo”.
5. Una empresa tiene cuatro escalas de salario numeradas consecutivamente del 1 al 4, además, tiene un programa de incentivos de acuerdo a la categoría y si el número de unidades producidas es mayor de 50. Si está en la categoría 1 se le da un incremento de sueldo equivalente al 5% de su salario, si está en la 2 del 7%, en la 3 del 10% y en la 4 del 15%; esto es por cada empleado. Se tiene la siguiente información: nombres, salario mensual, categoría y número de unidades producidas.

Ejercicios de repaso

6. En el minimercado de Apu hay una máquina dispensadora que tiene 4 productos etiquetados con los números 1, 2, 3 y 4, cada uno con un valor de \$500, \$800, \$300 y \$900, respectivamente. Defina un algoritmo que lea el número de producto que ingresa el usuario y que le muestre su precio.



Referencias

1. <https://ellibrodepython.com/>
2. <https://www.w3resource.com/index.php>
3. <https://realpython.com/>
4. <https://curriculumresources.edu.gh/wp-content/uploads/2025/01/>
5. <https://python-course.eu/>
6. <https://realpython.com/cheatsheets/python/>
7. <https://realpython.com/python-conditional-statements/>
8. <https://realpython.com/ref/best-practices/conditionals/>
9. <https://makecode.microbit.org/courses/cintro>
10. <https://makecode.microbit.org/courses>

Eso es todo amigos

